

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Zoltai Zétény Csongor
2026

**Szegedi Tudományegyetem
Informatikai Intézet**

**Gamifikált médiafogyasztás-követő
webalkalmazás blokklánc alapú jutalmazási
rendszerrel**

**(Gamified Media Consumption Tracking Web Application with
a Blockchain-Based Reward System)**

Szakedolgozat

Készítette:

**Zoltai Zétény
Csongor**

informatika szakos
hallgató

Témavezető:

Dr. Pflanzner Tamás

egyetemi adjunktus

Szeged
2026

Feladatkiírás

A szakdolgozat feladata egy olyan fullstack webalkalmazás megtervezése és megvalósítása, amely különböző típusú médiatartalmak egységes rendszerben történő kezelését teszi lehetővé.

A rendszer feladata, hogy nyílt forráskódú média-adatbázisok alkalmazásprogramozási felületeinek felhasználásával biztosítsa a médiatartalmak lekérdezését, valamint azok felhasználóhoz rendelt nyilvántartását. Az alkalmazás tegye lehetővé a felhasználók számára a megtekintett, folyamatban lévő, illetve később megtekintendő tartalmak rendszerezését és követését.

A fejlesztés során meg kell valósítani egy korszerű, reszponzív felhasználói felületet, továbbá egy szerveroldali rendszert, amely biztosítja az adatok tárolását, a felhasználók kezelését és a külső szolgáltatások integrációját.

A feladat részét képezi egy ösztönző jellegű jutalmazási mechanizmus kialakítása is, amely a felhasználói aktivitást támogatja. Ennek keretében a rendszer blokklánc-technológiára épülő megoldást alkalmaz, amely lehetővé teszi digitális tokenek megszerzését különböző tevékenységekhez (például tartalomfogyasztás, feladatok teljesítése) kapcsolódóan. A megszerzett tokenek a rendszerben elérhető testreszabási lehetőségek feloldására használhatók fel.

A rendszer tervezése során kiemelt szempont a modularitás és a bővíthetőség biztosítása, annak érdekében, hogy a későbbiekben további médiatípusok és funkcionális elemek integrálása is megvalósítható legyen.

Tartalmi összefoglaló

- **Téma megnevezése:**

Gamifikált médiafogyasztás-követő webalkalmazás blokklánc alapú jutalmazási rendszerrel

- **Megadott feladat megfogalmazása:**

A feladat egy olyan fullstack webalkalmazás megvalósítása, amely több nyílt média API integrálásával központi helyet biztosít a felhasználó médiatartalmainak kezelésére, nyomon követésére és értékelésére. A rendszer gamifikációs és blokklánc alapú jutalmazási réteget is tartalmaz, amely tokenekkel ösztönzi a felhasználói aktivitást, amelyek színsémák feloldására használhatók.

- **Megoldási mód:**

A médiafogyasztás-követő alkalmazás megalapozásához a rendszertervet, illetve adatbázis struktúrát hoztam létre, amelyek segítettek a program megfelelő megvalósításában. A rendszer egy Angular alapú frontendből, egy Express (Node.js) backendből, PostgreSQL adatbázisból, valamint egy Ethereum-kompatibilis okosszerződés rétegből épül fel. A frontend API-n keresztül kommunikál a backenddel, amely az üzleti logika, az adatkezelés, valamint a blokkláncal való kommunikáció megvalósításáért felel, míg a blokklánc réteg a token tranzakciók kezelését végzi. A megvalósított rendszert különböző metrikák mentén teszteltem.

- **Alkalmazott eszközök, módszerek:**

Angular (frontend SPA), Express (backend REST API), PostgreSQL (adatbázis), Hardhat (smart contract fejlesztés), TMDB API (külső adatforrás), MetaMask / Web3 integráció, JWT alapú autentikáció.

- **Elért eredmények:**

Egy működő webalkalmazás, amely lehetővé teszi a felhasználó számára filmek, sorozatok böngészését, a kívánt média követését, értékelését, emellett gamifikációs elem segítségével tokenszerzési lehetőséget biztosít, mellyel színsémát módosító témákat vásárolhat.

- **Kulcsszavak (4-6 darab):**

Gamifikáció, médiafogyasztás-követés, webalkalmazás, blokklánc, token alapú jutalmazás, API integráció

Tartalomjegyzék

Feladatkiírás	3
Tartalmi összefoglaló	4
Tartalomjegyzék	5
1. Bevezetés	7
1.1 Motiváció és problémafelvetés	7
1.2 Célkitűzés	7
2. Szakirodalmi és területi áttekintés	7
2.1 Médiafogyasztási platformok áttekintése	8
2.2 Hasonló rendszerek	9
2.2.1 Letterboxd	9
2.2.2 Trakt.tv	9
2.2.3 TMDb alapú alkalmazások	10
2.3 Gamifikáció szerepe	10
2.4 Blokklánc alapú jutalmazási rendszerek	11
2.5 Összehasonlító táblázat	12
3. Felhasznált technológiák	13
3.1 Frontend - Angular	13
3.2 Backend - Node.js és Express	14
3.3 Adatbázis - PostgreSQL	14
3.4 Külső API-k - TMDb	15
3.5 Blokklánc - Ethereum, Hardhat	16
3.6 Wallet integráció - MetaMask	17
4. Rendszerterv és architektúra	18
4.1 Áttekintő architektúra diagram	18
4.2 Frontend architektúra	20
4.3 Backend struktúra (API réteg)	21
4.4 Adatbázis modell	22
4.5 Blockchain integráció	24
5. Funkcionális specifikáció és implementáció	25
5.1 Felhasználói regisztráció / login	26
5.2 Média keresés és megtekintés	27

5.3 Médiakövetési lista kezelés	30
5.4 Profil adatmegjelenítés	32
5.5 Gamifikációs és jutalmazási rendszer	33
6. Tesztelés.....	36
6.1 Unit tesztek	36
6.2 API tesztelés	38
6.3 Teljesítményteszt	38
6.4 Blokklánc hálózatok összehasonlítása	41
7. Eredmények és értékelések	43
7.1 Rendszer működése	43
7.2 Teljesítmény.....	43
7.3 Korlátok	44
7.4 Továbbfejlesztési lehetőségek	44
8. Összefoglalás	45
Irodalomjegyzék	46
Nyilatkozat.....	48
Köszönetnyilvánítás.....	49

1. Bevezetés

1.1 Motiváció és problémafelvetés

Napjainkban a digitális tartalomfogyasztás rendkívül széttagolt. A különböző médiatípusok, például filmek, sorozatok, könyvek, videójátékok, más-más platformokon és alkalmazásokon keresztül érhetők el, ami megnehezíti a felhasználók számára a fogyasztott tartalmak nyomon követését.

E problémára megoldást nyújtanak a médiakövető alkalmazások, mint például a Goodreads [1], RAWG [2], Letterboxd [3] vagy a MyAnimeList [4], melyek lehetővé teszik a felhasználók számára az egyes médiatípusokhoz kapcsolódó tartalmak állapotának (megtekintett, folyamatban lévő, tervezett) rögzítését. A probléma ezekkel a rendszerekkel az, hogy egy médiatípus követését biztosítják, így minden különböző típusú tartalomhoz újabb platform szükséges, ami hosszabb távon nehézkessé és átláthatatlanná válhat.

A fentiek alapján indokolt egy olyan egységes rendszer kialakítása, amely képes több médiatípus egyidejű kezelésére és átlátható nyomon követésére.

1.2 Célkitűzés

A szakdolgozat célja egy olyan webalkalmazás megvalósítása, amely többféle médiatípus nyomon követését és értékelését teszi lehetővé egy egységes felületen, ezzel javítva a felhasználói élményt. A rendszer gamifikációs elemeket is tartalmaz, amelyek célja a felhasználói aktivitás ösztönzése. A felhasználók különféle feladatok teljesítésével tokeneket szerezhetnek, amelyek segítségével témákat oldhatnak fel. A feloldott témák az alkalmazás megjelenésének testreszabására használhatók fel.

2. Szakirodalmi és területi áttekintés

A digitális médiafogyasztás napjainkban elsősorban online és televíziós platformokon keresztül történik, amelyek különböző korlátokkal és felhasználói modellekkel rendelkeznek. A hagyományos televíziós szolgáltatások esetében a felhasználó

tartalomválasztási lehetősége erősen korlátozott, mivel a műsorstruktúra előre meghatározott.

Az internet és a streaming szolgáltatások elterjedésével a felhasználók számára jelentősen bővültek a tartalomfogyasztási lehetőségek, azonban ezzel párhuzamosan a piac erősen fragmentálttá vált [5]. A különböző szolgáltatók (például Netflix, Disney+, HBO Max) elkülönült ökoszisztémákban működnek, ami megnehezíti a fogyasztott tartalmak nyomon követését és rendszerezését.

E problémára különböző médiakövető platformok jelentek meg, amelyek célja a felhasználói médiafogyasztás strukturált kezelése. Ezek azonban jellemzően egy-egy médiatípusra specializálódnak, így nem biztosítanak egységes megoldást a különböző tartalomtípusok kezelésére.

2.1 Médiafogyasztási platformok áttekintése

A streaming platformok napjainkban a digitális médiafogyasztás egyik legelterjedtebb formáját jelentik, amelyek lehetővé teszik filmek, sorozatok és egyéb vizuális tartalmak gyors és felhasználóbarát elérését. Ezekben a rendszerekben a felhasználó egyszerűen kereshet a számára érdekes tartalmak között, majd azok azonnal megtekinthetők, ami jelentősen hozzájárult a hagyományos médiaszolgáltatási modellek átalakulásához.

A piac azonban erősen széttagolttá vált, mivel több, egymástól független szolgáltató, például Netflix, Disney+ vagy HBO Max, párhuzamosan működik, eltérő tartalomkínálattal és előfizetési modellekkel. Ez azt eredményezi, hogy a felhasználóknak több platformot is használniuk kell a különböző médiatartalmak elérése során, ami hosszabb távon átláthatatlanná és költségessé teszi a médiafogyasztást [5].

E problémára válaszul jelentek meg a médiakövető platformok [1][2][3], amelyek célja a felhasználói médiafogyasztás centralizált nyilvántartása és kezelése. Ezek az alkalmazások lehetőséget biztosítanak a tartalmak állapotának rögzítésére, azonban jellemzően egy-egy médiatípusra fókuszálnak, így nem nyújtanak egységes megoldást a különböző típusú médiafogyasztás kezelésére.

Ez a probléma teremti meg az igényt egy olyan egységes rendszerre, amely több médiatípus kezelését is képes támogatni, és a felhasználói élményt egy központi felületen keresztül biztosítja.

2.2 Hasonló rendszerek

A fent említett megoldások nem biztosítanak teljes körű megoldást a médiakövetés széttagoztságának problémájára. Ebben a fejezetben ezeket a már létező platformokat fogom vizsgálni, amely lehetővé teszi a hiányosságok azonosítását, valamint alapot ad a saját rendszer tervezési döntéseinek indoklásához.

2.2.1 Letterboxd

A Letterboxd egy filmkövetésre specializálódott webes platform, amely felhasználóbarát kezelőfelülettel és aktív közösségi bázissal rendelkezik. A rendszer központi eleme a felhasználók által létrehozott vélemények, értékelések megosztása, amelyek hozzájárulnak a filmek közösségi alapú felfedezéséhez [6].

Az oldal TMDb adatbázisra építve biztosítja a filmekhez kapcsolódó alapvető információkat [7]. Ez lehetővé teszi, hogy a felhasználók naprakész adatokat kapjanak a filmekről.

Funkcionális szempontból a Letterboxd lehetőséget biztosít filmek megtekintésként való jelölésére, valamint várólisták létrehozására. Emellett a felhasználók egymást követhetik, illetve kommentek és értékelések formájában interakcióba léphetnek más felhasználókkal, ami erős közösségi réteget ad a platformnak.

A rendszer erőssége a közösségi aktivitás és a tartalmak felfedezésének támogatása, azonban kizárólag filmek érhetők el, így nem biztosít egységes megoldást más médiatípusok kezelésére. Emellett nem rendelkezik olyan jutalmazási vagy motivációs rendszerrel, amely a felhasználói aktivitást explicit módon ösztönözné, például token alapú vagy feloldható tartalmak formájában.

2.2.2 Trakt.tv

A Trakt.tv egy film- és sorozatkövetésre specializált webalkalmazás, amely elsősorban a felhasználói megtekintési előzmények nyomon követésére helyezi a hangsúlyt. A rendszer egyik kiemelkedő funkciója, hogy különböző médialejátszókkal és szolgáltatásokkal integrálva képes automatikusan naplózni a megtekintett tartalmakat, így a felhasználónak nem szükséges manuálisan rögzítenie az előrehaladását [8].

Az alkalmazás egy központi felületen gyűjti össze a felhasználó megtekintési előzményeit, függetlenül attól, hogy milyen eszközön vagy platformon történt a tartalomfogyasztás. Emellett lehetőséget biztosít értékelések és vélemények megadására, valamint több platformon elérhető, szinkronizált működést kínál.

A rendszer erőssége az automatizált médiakövetés és az integrációs lehetőségek, ugyanakkor bizonyos esetekben az automatikus naplózás pontatlanságokat eredményezhet, amely manuális korrekciót igényelhet. Továbbá a platform nem tartalmaz kifejezett gamifikációs elemeket, így a felhasználói aktivitás ösztönzése elsősorban nem játékosított módon történik.

2.2.3 TMDb alapú alkalmazások

A TMDb egy népszerű, közösség által épített és szerkeszthető online adatbázis, amely filmek és televíziós műsorok metaadatait tartalmazza, és széles körben használt alkalmazásprogramozási felületet (API) biztosít külső rendszerek számára [9].

A szolgáltatás egyik legnagyobb előnye a kiterjedt és folyamatosan frissített adatbázis, amely lehetővé teszi, hogy a fejlesztők saját alkalmazásaikban naprakész médiainformációkat jelenítsenek meg anélkül, hogy külön adatbázist kellene fenntartaniuk.

A TMDb API integrációjával elegendő a médiaazonosítók vagy erőforrás-hivatkozások tárolása, míg a részletes tartalmi adatok dinamikusan lekérhetők [10]. Ugyanakkor az API elsősorban adatforrásként működik, és nem biztosít komplex felhasználói logikát vagy gamifikációs elemeket, így ezek megvalósítása a ráépülő rendszerek feladata.

A vizsgált adatok rávilágítanak ezen projektek gyengeségeire, melyek a saját rendszeremben implementálva lesznek. Ezek az absztrakciók, az egynél több médiatípus támogatása, a gamifikáció, és a jutalmazási rendszer vizualizációs elemekkel

2.3 Gamifikáció szerepe

Napjainkban sok szolgáltatás biztosít olyan funkcionalitást, amely egy idő után monotonná, repetitívvé válik, így a felhasználó motivációját veszítheti igénybevételük során.

Erre megoldást nyújthat a gamifikáció [11]. A gamifikáció olyan módszer, amely játékmekanikákat alkalmaz nem játék jellegű környezetben a felhasználói motiváció és elköteleződés növelése érdekében [12]. Segítségével olyan jutalmazási rendszert lehet az adott funkcionalitás köré építeni, ami ösztönzi a felhasználót a további használatra. Ilyen elemek lehetnek például a feladatok teljesítéséhez kötött szintek, jelvények, jutalmak, kihívások, egy új absztrakciót nyújtva a rendszerhez, kiegészítve az oldal nyújtotta élményt.

Ez a megközelítés nemcsak a monotonitást tudja elkerülni, hanem motiválni tudja a felhasználót az adott célra is. A szakdolgozatomban gamifikációs megközelítéssel ösztönzőm a felhasználókat médiafogyasztás nyomon követésére feladatok teljesítésén keresztül. A feladat nehézségtől függően bizonyos mennyiségű tokenre tesz szert a felhasználó, amit megadott összegért el tud költeni az alkalmazás megjelenését módosító témákra.

2.4 Blokklánc alapú jutalmazási rendszerek

A blokklánc technológia egy decentralizált adatbázist biztosít, amelyet a hálózat résztvevői tartanak fenn [13]. A rendszer átláthatóságát az biztosítja, hogy a rögzített tranzakciók nyilvánosan ellenőrizhetők, míg a biztonságát az adja, hogy az adatblokkok utólagos módosítása jelentős számítási erőforrást igényel. A rögzített adatok tartós megőrzése hozzájárul a rendszer megbízható működéséhez.

A jutalmazási rendszerek esetében a blokklánc előnye, hogy könnyen lehet egy robusztus vásárlási réteget megvalósítani, ahol a tokenek kiosztása és felhasználása átlátható módon történik. Emellett a tranzakciók ellenőrizhetők, így csökkenthető a rendszerrel való visszaélés lehetősége.

A blokklánc technológiát gyakran alkalmazzák digitális jutalmazási rendszerekben, amelyek jellemzően token alapú ösztönző mechanizmusokra épülnek. Ezek a megoldások leggyakrabban decentralizált alkalmazások (dApp-ok) formájában jelennek meg [14], amelyek különböző módokon valósítják meg a jutalmazást.

Egyes rendszerek kriptovaluták használatával biztosítanak jutalmakat, ahol a felhasználók különböző tevékenységekért cserébe digitális pénzben részesülnek. Más megközelítések nem helyettesíthető tokeneket (NFT-eket) alkalmaznak, amelyek egyedi digitális jutalmakat vagy gyűjthető elemeket képviselnek.

Ezek az alkalmazások gyakran gamifikációs elemekkel egészülnek ki, például pontokkal, ranglistákkal vagy jutalmazási rendszerekkel, amelyek célja a felhasználói aktivitás és elköteleződés növelése.

A blokklánc alapú megközelítés előnye a hagyományos adatbázis-alapú megoldásokkal szemben, hogy a tranzakciók utólag nem módosíthatók, ami lehetővé teszi a rendszer működésének független ellenőrzését. Ez növeli az átláthatóságot, valamint csökkenti a központi szerverbe vetett bizalom szükségességét. Emellett lehetőséget biztosít a rendszer későbbi bővítésére és más blokklánc-alapú szolgáltatásokkal való integrációra.

A szakdolgozatban bemutatott rendszerben a felhasználók különböző feladatok teljesítéséért tokeneket kapnak, amelyeket az alkalmazás megjelenésének testreszabására használhatnak fel.

2.5 Összehasonlító táblázat

Funkció	Letterboxd	Trakt.tv	Saját rendszer
Támogatott médiatípus	Film	Film, sorozat	Film, sorozat
Követés típusa	Manuális	Félig automatikus, integráció alapú	Manuális, feladat alapú követés
Gamifikációs elemek	Kihívások (közösségi alapú)	Nincs	Aktivitás alapú feladatok
Jutalmazási logika	Nincs valódi jutalmazási rendszer	Nincs	Token alapú jutalom
Motivációs rendszer	Közösségi aktivitás	Statisztikai követés	Tokenből vásárolható témák
Cél	Közösségi filmnapló	Egységes médiakövetés és szinkronizáció	Egységes gamifikált médiakövetés

2.1 táblázat - Hasonló rendszerek összehasonlítása

A 2.1. táblázat alapján megfigyelhető, hogy a vizsgált rendszerek elsősorban egy adott funkcionális területre specializálódtak, és nem biztosítanak egységes, több szempontot lefedő megoldást a médiafogyasztás kezelésére. A Letterboxd erőssége a közösségi alapú értékelés és interakció, míg a Trakt.tv inkább a megtekintési előzmények automatikus szinkronizációjára és statisztikai feldolgozására fókuszál.

E rendszerekhez képest a bemutatott saját rendszer célja egy egységesített, több médiatípust kezelni képes platform kialakítása, amely a felhasználói aktivitást nyomon követi, és egy strukturált jutalmazási mechanizmuson keresztül ösztönzi is.

3. Felhasznált technológiák

Ebben a fejezetben a rendszer megvalósítása során alkalmazott technológiák kerülnek bemutatásra.

3.1 Frontend - Angular

Az Angular [\[15\]](#) egy nyílt forráskódú, TypeScript [\[16\]](#) alapú frontend keretrendszer webalkalmazások fejlesztéséhez, amelyet a Google fejleszt és tart karban. 2016-ban mutatták be, azóta a modern egyoldalas webalkalmazások (Single Page Application - SPA) fejlesztéséhez alkalmazzák.

Az SPA [\[17\]](#) megközelítés lényege, hogy az alkalmazás egyetlen betöltéssel működik, és a felhasználói interakciók során dinamikusan frissíti a felületet, így nincs szükség az oldalak teljes újratöltésére.

A keretrendszer legfontosabb jellemzői közé tartozik a komponens alapú architektúra, amely lehetővé teszi a felhasználói felület kisebb, újrafelhasználható egységekre bontását. Emellett támogatja a kétirányú adatkötést, amely biztosítja, hogy a felhasználói felület és az alkalmazás állapota kölcsönösen szinkronban maradjon. A függőséginjektálás révén az alkalmazás komponensei között laza csatolás valósítható meg, ami elősegíti a rendszer modularitását és a függőségek hatékony kezelését. Továbbá az Angular széles körű beépített eszközkészlettel rendelkezik, amely csökkenti a külső könyvtárak használatának szükségességét. Emellett beépített routing megoldást kínál, amely lehetővé teszi az alkalmazáson belüli navigáció kezelését különálló oldalak újratöltése nélkül.

A keretrendszer további előnye, hogy erősen strukturált fejlesztési modellt biztosít, amely megkönnyíti a nagyobb alkalmazások karbantartását és bővíthetőségét. A TypeScript használata növeli a kód megbízhatóságát és átláthatóságát, mivel statikus típusellenőrzést tesz lehetővé.

A frontend megvalósítása során több modern JavaScript keretrendszer is szóba jöhetett, például React vagy Vue. Az Angular mellett azért esett a választás, mert jól támogatja a komplex, több nézetből álló webalkalmazások fejlesztését, valamint lehetőséget biztosít egy moduláris, könnyen bővíthető frontend architektúra kialakítására. Emellett a beépített szolgáltatások (például HTTP kliens és routing) csökkentik a külső függőségek szükségességét.

3.2 Backend - Node.js és Express

A Node.js egy nyílt forráskódú, platformfüggetlen JavaScript futtatókörnyezet, amely a böngészőn kívül, a V8 JavaScript motor segítségével fut [\[18\]](#). Eseményvezérelt, nem blokkoló I/O modellt alkalmaz, amely lehetővé teszi, hogy a rendszer ne várja meg a hosszabb műveletek (például adatbázis lekérdezések) befejezését, hanem párhuzamosan más feladatokat is kezeljen. Ennek köszönhetően hatékonyan támogatja a nagy számú egyidejű kérés kezelését, ami különösen előnyös webalkalmazások és API-k fejlesztése során.

Az Express egy minimális és rugalmas Node.js webes keretrendszer, amely leegyszerűsíti a szerveroldali alkalmazások és REST API-k fejlesztését [\[19\]](#). Middleware alapú architektúrája lehetővé teszi a kérések és válaszok feldolgozásának rétegezését, így a közös funkciók (például autentikáció, logolás) külön kezelhetők. A beépített routing rendszer segítségével az alkalmazás végpontjai strukturáltan és könnyen kezelhetően definiálhatók. Az Express jól integrálható különböző adatbázis-megoldásokkal, valamint frontend keretrendszerekkel, így széles körben alkalmazott backend technológia.

A backend fejlesztése során alternatív megoldásként felmerültek más technológiák is, például a Spring Boot alapú megközelítés. A Node.js és Express használata mellett döntöttem, mivel hatékonyan támogatják a REST API [\[20\]](#) alapú architektúrák kialakítását, valamint gyors fejlesztést tesznek lehetővé. A JavaScript alapú egységes technológiai stack (frontend és backend) egyszerűbb fejlesztést és könnyebb karbantarthatóságot biztosít.

3.3 Adatbázis - PostgreSQL

A PostgreSQL egy nyílt forráskódú, relációs adatbázis-kezelő rendszer, amely az SQL nyelvet kibővítve fejlett adatkezelési lehetőségeket biztosít [\[21\]](#). A rendszer fejlesztése

az 1980-as években indult a Berkeley Kaliforniai Egyetemen a POSTGRES projekt keretében, és azóta is folyamatos fejlesztés alatt áll. A PostgreSQL megbízható működése és széles körű funkcionalitása miatt napjainkban is az egyik legelterjedtebb adatbázis-kezelő rendszer.

A relációs adatbázis olyan típusú adatbázis, amely az adatokat egymással kapcsolatban álló táblákba rendezi a relációs adatmodell alapján, amelyek sorok és oszlopok szerint strukturáltak [22]. Elsősorban olyan esetekben használatos, ha az adatpontosság, konzisztencia, nyomon követhetőség elengedhetetlen.

A projekt során felhasználói adatokat, Web3 pénztárcákhoz tartozó információkat, valamint médiatartalmakhoz kapcsolódó adatokat szükséges tárolni. Ezek között szoros kapcsolatok állnak fenn, ezért kiemelten fontos a jól definiált adatstruktúra és a kapcsolatok kezelése. A relációs modell nemcsak átláthatóvá teszi az adatstruktúrát, hanem a külső API adatainak tárolását is megkönnyíti.

Alternatív megoldásként felmerült a MySQL, a MariaDB és a MongoDB használata is. A MySQL széles körben elterjedt, azonban bizonyos esetekben kevesebb fejlett funkcionalitást kínál. A MariaDB a MySQL továbbfejlesztett változata, amely bővített lehetőségeket biztosít. A MongoDB dokumentumorientált adatbázisként nagy mennyiségű adat gyors kezelésére alkalmas, azonban kevésbé támogatja az összetett relációk kezelését [23].

A PostgreSQL választását indokolta a komplex lekérdezések hatékony támogatása, a megbízható működés, valamint az, hogy jól illeszkedik a Node.js alapú backend környezethez. Emellett a korábbi tapasztalatok is hozzájárultak ahhoz, hogy a projektben ezt az adatbázis-kezelő rendszert alkalmaztam.

3.4 Külső API-k - TMDB

Az Application Programming Interface, magyarul alkalmazásprogramozási felület, olyan köztes réteg, amely lehetővé teszi különböző rendszerek közötti kommunikációt [24]. A webalkalmazások esetében elterjedt megoldás a REST architektúrát követő API használata, amely HTTP protokollon keresztül biztosítja az adatok lekérését és továbbítását [25].

A rendszerben a TMDB API (The Movie Database API) kerül felhasználásra, amely egy széles körben alkalmazott online szolgáltatás filmekhez és sorozatokhoz kapcsolódó metaadatok elérésére [26]. Az API segítségével különböző lekérdezések

hajthatók végre, például tartalomkeresés, részletes adatok lekérése vagy ajánlások megjelenítése.

A szolgáltatás igénybevételéhez regisztráció szükséges, amely során a feltételek elfogadásával szert tehet a felhasználó egy egyedi API kulcsra. Ez a kulcs azonosítja a kéréseket és biztosítja a hívások szabályozását. Az alkalmazás HTTP kéréseken keresztül megfelelő végpontok és paraméterek segítségével kapja vissza a strukturált választ.

Film- és sorozatadatok esetében a TMDb API választását a könnyű integrálhatóság, a kiterjedt és folyamatosan frissített adatbázis, valamint az indokolta, hogy széles körben alkalmazott és megbízható forrásnak tekinthető.

3.5 Blokklánc - Ethereum, Hardhat

A blokklánc egy decentralizált, elosztott adatbázis [13], amely időrendben egymáshoz kapcsolt blokkok sorozataként tárolja az adatokat. Minden blokk tranzakciókat tartalmaz, valamint egy kriptográfiai hivatkozást az előző blokkra, ezáltal láncszerű struktúrát alkotva [27]. Ennek a kialakításnak köszönhetően az adatok utólagos módosítása rendkívül nehéz, mivel egy blokk megváltoztatása a teljes lánc módosítását igényelné.

A blokklánc alapú rendszerekben a tranzakciók blokkokba rendezve kerülnek rögzítésre, amelyeket a hálózat résztvevői, az úgynevezett node-ok kezelnek. Ezek a csomópontok a blokklánc másolatát tárolják, ellenőrzik a tranzakciók érvényességét, és biztosítják az adatok terjesztését a hálózatban. A blokkok érvényesítését konszenzusmechanizmus biztosítja. A korábbi rendszerekben alkalmazott Proof of Work esetében ez számítási erőforrásokon alapuló miner-ek közti verseny volt, míg az Ethereum jelenlegi működése során a Proof of Stake modell kerül alkalmazásra, ahol a validátorok a lekötött tőkéjük arányában vesznek részt az új blokkok létrehozásában és hitelesítésében. A kiválasztott miner vagy validator által javasolt blokkot a hálózat többi node-ja ellenőrzi, majd konszenzus alapján a blokklánchoz csatolja.

Az okosszerződések (smart contractok) a blokkláncon futó programok, amelyek előre meghatározott feltételek teljesülése esetén automatikusan végrehajthatók. Ezek a programok módosíthatatlanok, és központi szerepet játszanak a decentralizált alkalmazások működésében. A bemutatott rendszerben az okosszerződések felelősek a felhasználók jutalmazási logikájának megvalósításáért.

Az Ethereum egy nyílt forráskódú blokklánc platform [28], amely nemcsak digitális pénzügyi tranzakciók végrehajtását teszi lehetővé, hanem támogatja az

okosszerződések futtatását is. Ennek köszönhetően alkalmas decentralizált alkalmazások (dApp-ok) fejlesztésére [29], valamint saját tokenek létrehozására. A platform működésének alapját az Ethereum Virtual Machine (EVM) adja, amely biztosítja az okosszerződések futtatási környezetét és a determinisztikus végrehajtást a hálózat minden csomópontján. Az Ethereum széles körű elterjedtsége és fejlett ökoszisztémája stabil alapot biztosít blokklánc-alapú rendszerek megvalósításához.

A Hardhat egy fejlesztői környezet, amely az Ethereum-alapú alkalmazások fejlesztését támogatja [30]. Lehetővé teszi okosszerződések írását, tesztelését és üzembe helyezését, valamint fejlett hibakeresési eszközöket biztosít. A Hardhat emellett lehetőséget biztosít lokális blokklánc környezet létrehozására is, amely jelentősen megkönnyíti a fejlesztési és tesztelési folyamatokat valós tranzakciók költsége nélkül.

A blokklánc technológia alkalmazása mellett elsősorban annak decentralizált működése, átláthatósága és a tranzakciók módosíthatatlansága szolgált, amelyek különösen fontosak a jutalmazási rendszer megbízható működése szempontjából. Az okosszerződések lehetővé teszik a jutalmazási logika automatizált és ellenőrizhető megvalósítását, ezáltal csökkentve a központi rendszerbe vetett bizalom szükségességét.

Alternatív megoldásként felmerült a hagyományos, szerveroldali jutalmazási rendszer alkalmazása is [31], amely egyszerűbben implementálható, azonban nem biztosítja a tranzakciók átláthatóságát és módosíthatatlanságát. Továbbá más blokklánc platformok, például a Solana vagy a Binance Smart Chain használata is lehetséges lett volna, azonban az Ethereum széles körű elterjedtsége, fejlett fejlesztői eszköztára és stabil ökoszisztémája miatt bizonyult a legmegfelelőbb választásnak. A fejlesztés során az Ethereum ökoszisztémájához a Hardhat keretrendszer került kiválasztásra, mivel hatékony támogatást nyújt az okosszerződések fejlesztéséhez, teszteléséhez és telepítéséhez.

3.6 Wallet integráció - MetaMask

A Web3 digitális pénztárca egy szoftver vagy hardver eszköz, amely segítségével a felhasználók kezelhetnek és használhatnak kriptovalutákat, tokeneket (pl. NFT-eket), valamint kapcsolatba léphetnek decentralizált alkalmazásokkal (dApp-okkal) több blokklánc hálózaton át [32]. A felhasználók egy privát és egy publikus kulcs segítségével működnek. A privát kulcs a tranzakciók aláírására szolgál, amely igazolja a felhasználó jogosultságát a blokkláncon tárolt digitális értékei felett, míg a publikus kulcs (illetve az

abból származtatott cím) a fogadó fél azonosítására használható. A tranzakció aláírás egy digitális reprezentációja az eseménynek, azt jelzi, hogy a felhasználó elfogadja a tranzakciót.

A MetaMask egy letétkezelő nélküli kriptopénztárca, ami a felhasználónak biztonságos elérést biztosít blokklánc alapú alkalmazásokhoz és tokenekhez [33]. Elérhető böngésző bővítményként, telefonon pedig applikációként. Támogatja elsődlegesen az Ethereum ökoszisztémát, és az ERC-20 [34], ERC-721 [35], ERC-1155 [36] tokeneket is. A felhasználói azonosítás a publikus kulcs alapján történik, amelyet a pénztárcában a valuták mellett lehet megtalálni [37]. A tranzakció jóváhagyása engedély egy decentralizált applikációnak, hogy elérje, és mozgassa az adott tokent a felhasználó pénztárcájából [38]. Ez azért szükséges, mert például egy utalás gombbal még nincs engedélye az applikációnak az összeg eléréséhez, el kell fogadnia a felhasználónak a MetaMask-on belül, így biztonságosabbá téve a tranzakciókat.

A webalkalmazás és a pénztárca közötti kapcsolatot a böngészőn keresztül valósul meg. A felhasználó kezdeményezi a pénztárca csatlakoztatását, amely során engedélyt ad az alkalmazás számára a cím elérésére és a tranzakciók aláírására. A tranzakciók jóváhagyása minden esetben a felhasználó által, a pénztárca felületén történik.

A projektben a MetaMask szerepe a jutalmazási rendszerhez kapcsolódás, a tokenek lekérdezése és kezelése, valamint az okosszerződéssel való kommunikáció biztosítása, mivel a tranzakciók aláírása a pénztárcán keresztül történik.

A MetaMask választását indokolja, hogy világszerte használt, megbízható szolgáltatás, ami könnyedén csatlakoztatja a web3 pénztárcákat a decentralizált alkalmazásokhoz. Emellett a frontendbe egyszerű az integrációja, és Ethereum kompatibilis.

4. Rendszerterv és architektúra

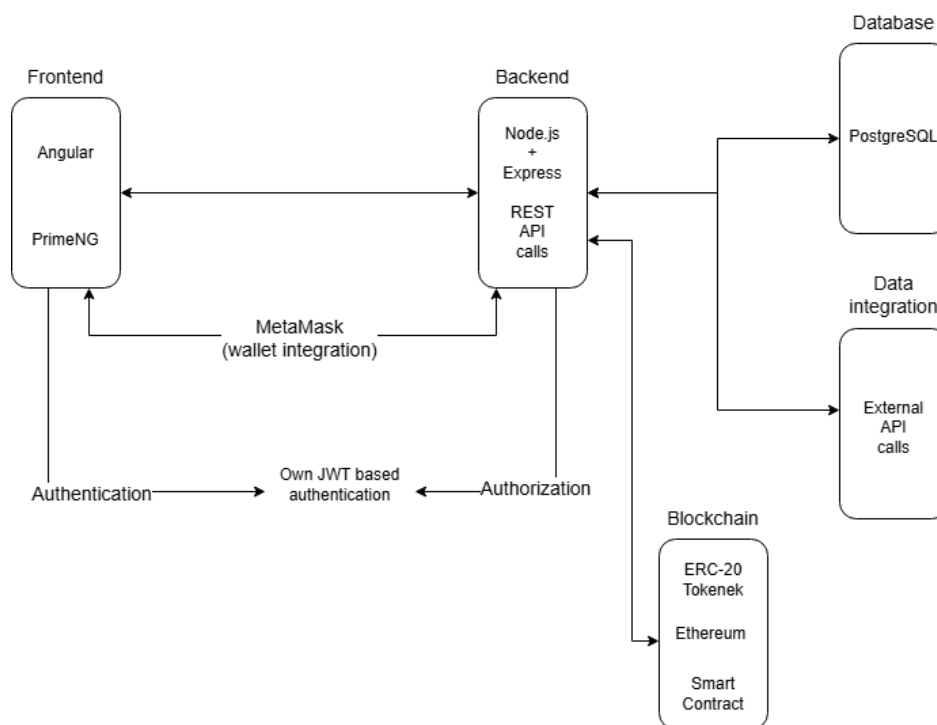
Ebben a fejezetben a rendszer felépítésének és működésének áttekintése kerül bemutatásra. A fejezet ismerteti az alkalmazás főbb architekturális rétegeit, azok kapcsolatát, valamint az egyes komponensek szerepét a rendszer működésében.

4.1 Áttekintő architektúra diagram

A rendszer három fő rétegre tagolható: frontend, backend és blokklánc réteg, ahogyan a 4.1. ábrán látható. A rétegek egymástól elkülönítve, de egymással szoros kapcsolatban működnek. A kommunikáció HTTP alapú REST API hívásokon keresztül valósul meg.

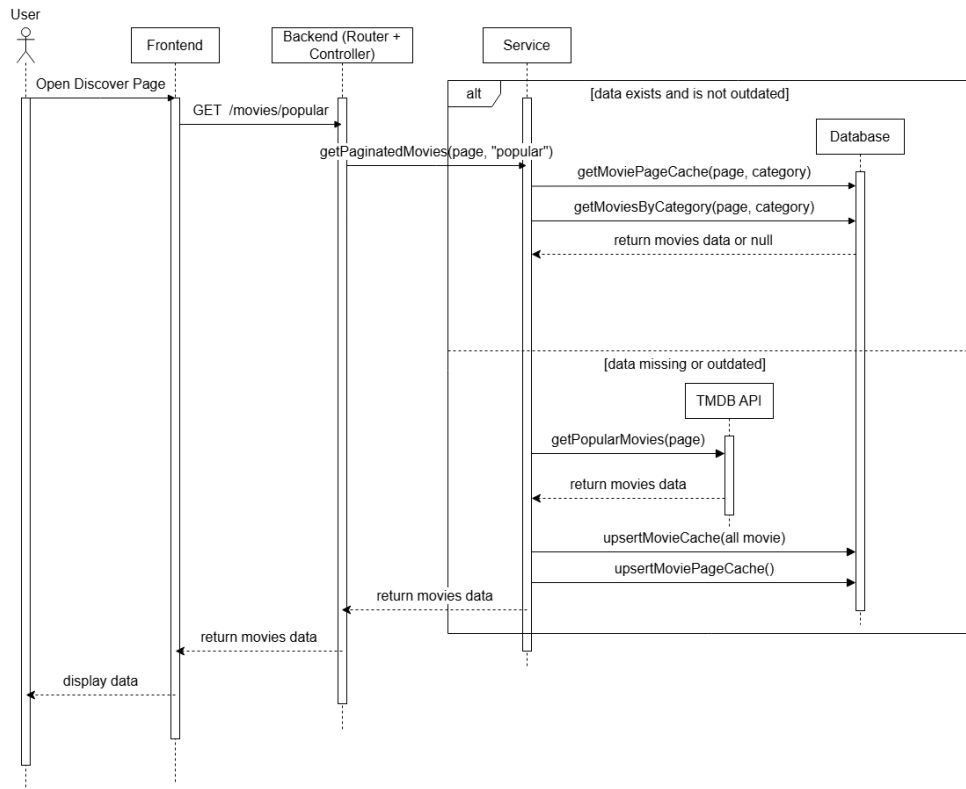
A frontend réteg felel a felhasználói felület megjelenítéséért és a felhasználói interakciók kezeléséért. A backend réteg biztosítja az üzleti logika megvalósítását, kezeli az adatbázis műveleteket, valamint közvetítő szerepet tölt be a külső szolgáltatások és a kliensoldal között.

A blokklánc réteg a rendszer gamifikációs funkcióit valósítja meg okoszerződések segítségével, míg az adatbázis réteg a maradandó adatok strukturált tárolásáért felel.



4.1 ábra - Projekt architektúra diagram

Egy tipikus adatfolyamat során a felhasználó a frontend felületen indít egy műveletet, amely a backendhez kerül továbbításra, ahogyan a 4.2. ábrán látható. A backend szükség esetén lekérdezi a TMDB API-t vagy hívást indít az okoszerződésnek, majd az eredményt feldolgozva visszaküldi a frontend számára megjelenítésre.



4.2 ábra - Adatfolyam diagram

4.2 Frontend architektúra

A frontend architektúra az Angular komponensalapú felépítésére épül, amelynek célja a moduláris, jól karbantartható és skálázható rendszer kialakítása.

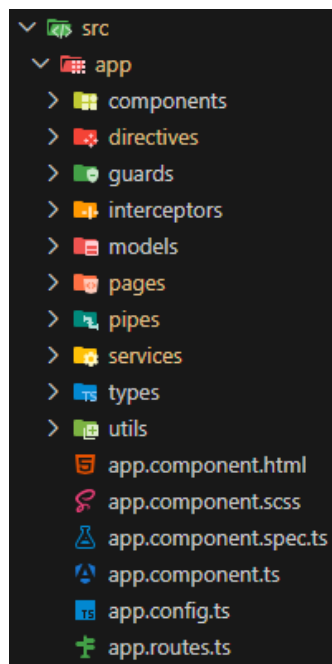
Az alkalmazás felépítése oldalakra és komponensekre tagolódik. Az egyes nézetek különálló route-okhoz vannak rendelve, amelyek közötti navigációt az Angular Routing biztosítja. Az oldalakon belüli logikai egységek komponensekbe vannak szervezve, amelyek hierarchikus struktúrában épülnek fel, lehetővé téve az újrafelhasználhatóságot és az átlátható felépítést.

A backenddel való kommunikáció egy külön service rétegen keresztül történik, amely kezeli a HTTP kéréseket és a válaszok feldolgozását. Az interceptorok egységes módon egészítik ki a kéréseket, például autentikációs adatok hozzáadásával vagy hibakezeléssel.

Az alkalmazás működésének szabályozásában a guard-ok és egyéb Angular mechanizmusok játszanak szerepet. A guard-ok az útvonalakhoz való hozzáférést kontrollálják, míg a pipe-ok az adatok megjelenítés előtti átalakítását végzik. A direktívák az interaktív viselkedés megvalósítását támogatják.

A felhasználói felület kialakításában a PrimeNG komponenskönyvtár, valamint saját stílusmegoldások kerültek alkalmazásra. A konzisztens megjelenés érdekében a stílusok strukturált módon, újrafelhasználható elemek (például design tokenek és SCSS mixinek) segítségével lettek kialakítva.

A 4.3. ábrán látható kliensoldali architektúra lehetővé teszi a frontend réteg jól strukturált működését, valamint megkönnyíti az alkalmazás későbbi bővítését és karbantartását.



4.3 ábra - Kliensoldali architektúra

4.3 Backend struktúra (API réteg)

A backend réteg feladata az üzleti logika megvalósítása, valamint a kommunikáció biztosítása a frontend, az adatbázis és a külső szolgáltatások között. A rendszer működéséhez szükséges, nem kliensoldali műveletek ezen a rétegen kerülnek feldolgozásra, megfelelő adatkezeléssel és hibakezeléssel kiegészítve.

A kódrendszer REST architektúrát követ, amelyben az egyes műveletek (GET, POST, PUT, DELETE) külön végpontokon keresztül érhetők el, ezáltal biztosítva az átlátható és karbantartható felépítést. A routing réteg határozza meg az egyes URL-ekhez tartozó végpontokat, és továbbítja a beérkező kéréseket a megfelelő controller felé.

A controller réteg felelős a beérkező kérések feldolgozásáért, a paraméterek kezeléséért, valamint a válasz összeállításáért. Innen kerül meghívásra a service réteg,

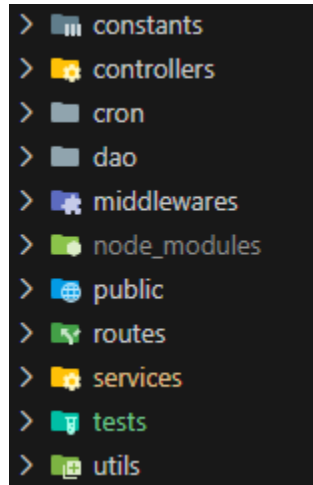
amely az üzleti logikát tartalmazza. A service réteg végzi el a szükséges adatfeldolgozást, majd a DAO (Data Access Object) rétegen keresztül kommunikál az adatbázissal. A DAO réteg valósítja meg az adatbázis műveleteket, és az eredményt visszajuttatja a magasabb rétegek felé.

A backend működése során a frontend HTTP kéréseket küld a szerver felé, amelyek feldolgozása után a rendszer strukturált válaszokat (JSON formátumban) ad vissza, vagy hiba esetén megfelelő státuszkódot küld.

A rendszer további kiegészítő elemei közé tartoznak a konstans értékeket tartalmazó modulok, amelyek elősegítik a kód egységességét, valamint az újrafelhasználható segédfüggvényeket tartalmazó util réteg. A middleware-ek a kérések feldolgozásának egyes lépései közé illesztve biztosítanak közös funkcionalitásokat, például autentikációt vagy hibakezelést.

A backendben cron alapú időzített feladatok is alkalmazásra kerülnek, amelyek lehetővé teszik bizonyos műveletek automatikus, időzített végrehajtását.

A 4.4. ábrán látható alkalmazott architektúra előnye, hogy jól strukturált, átlátható, valamint megkönnyíti a rendszer későbbi bővítését és karbantartását.



4.4 ábra - Szerveroldali architektúra

4.4 Adatbázis modell

Az adatbázis az alkalmazás egyik központi eleme, amely biztosítja az adatok tartós tárolását, valamint azok strukturált és hatékony kezelését. A megfelelően megtervezett adatbázis modell lehetővé teszi az alkalmazás funkcióinak megbízható működését, valamint támogatja a későbbi bővíthetőséget és karbantarthatóságot. A rendszerben

relációs adatbázis kerül alkalmazásra, amely táblák és azok közötti kapcsolatok segítségével modellezi az alkalmazásban kezelt adatokat.

Az adatbázis modell több logikai egységre bontható, ezek a felhasználói adatok kezelése, médiatartalmak gyorsítótárazása, felhasználói aktivitás és visszajelzések, valamint a gamifikációs rendszer (questek és jutalmak). Az egyes egységek egymással kapcsolatban állnak, amelyet idegen kulcsok biztosítanak.

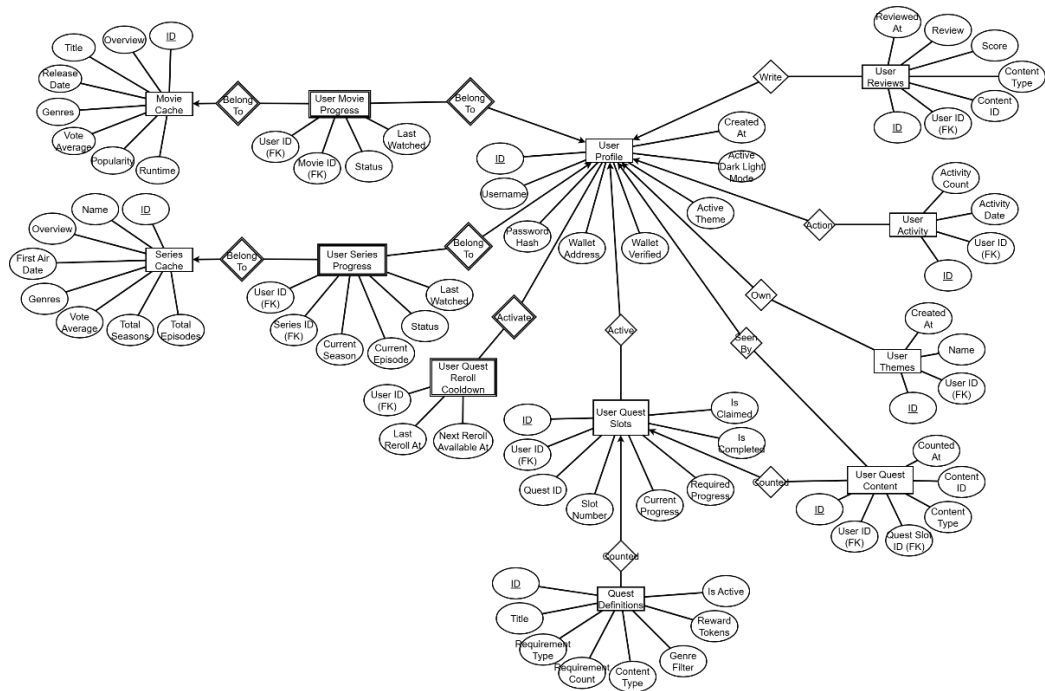
A rendszer központi eleme a UserProfile tábla, amely a felhasználók alapadatait tartalmazza, például felhasználónév, jelszó hash, valamint a blokklánc pénztárca címe. Ehhez kapcsolódnak további táblák, a UserThemes, ami a felhasználói testreszabási adatokat tárolja, a UserActivity, ami az aktivitás nyomon követéséhez szükséges adatokat tartalmazza, valamint a UserReviews, amely a felhasználók értékeléseit és véleményeit tárolja az egyes médiatartalmakhoz kapcsolódóan.

A médiatartalmak adatai a felhasználói profiloktól elkülönítve kerülnek tárolásra. A kapcsolódó információk nem teljes egészében kerülnek mentésre, hanem cache mechanizmuson keresztül, elsősorban a MovieCache és SeriesCache táblákban. Ezek a médiatartalmak részletes adatait tárolják ideiglenesen a külső API hívások számának csökkentése és a gyorsabb válaszidő biztosítása érdekében.

A rendszer emellett külön cache táblákat alkalmaz az oldalszintű listák és kategóriák kezelésére is, például a MoviePageCache, SeriesPageCache és GenreCache táblák segítségével. Ez lehetővé teszi a gyakran lekérdezett adatok hatékonyabb kiszolgálását és csökkenti a külső szolgáltatások terhelését.

A gamifikációs logika több egymással összefüggő táblában kerül megvalósításra. A QuestDefinitions tábla az elérhető feladatok definícióit tárolja, a UserQuestSlots a felhasználó aktuális feladatait tartalmazza, a UserQuestContent pedig a teljesítéshez kapcsolódó tartalmakat menti el. A UserQuestRerollCooldown tábla az új feladatok generálásának időbeli korlátozásához szükséges adatokat tárolja.

Az egyes táblák közötti kapcsolatokat idegen kulcsok biztosítják, amelyek garantálják az adatok konzisztenciáját. Egy felhasználóhoz például több aktivitás, értékelés és feladat is tartozhat, amely egy-egy, egy-több kapcsolatként jelenik meg az adatmodellben. Ezt a felépítést a 4.5. ábra szemlélteti.



4.5 ábra - Egyed-kapcsolat diagram

Az adatbázis tervezése során fontos szempont volt a normalizáció elveinek követése, amely csökkenti az adatredundanciát és biztosítja az adatok konzisztenciáját. Emellett a cache táblák alkalmazása tudatos döntés volt a teljesítmény optimalizálása érdekében, mivel a külső API hívások számát csökkenti. A struktúra moduláris felépítése lehetővé teszi új médiatípusok vagy funkciók későbbi integrálását.

4.5 Blockchain integráció

A blockchain réteg a gamifikációs rendszer megvalósítása során a tokenek kezeléséért, a tranzakciók végrehajtásáért és a jutalmazási logika biztosításáért felel. Ez a logika az átláthatóság és a visszakövethetőség érdekében blokklánc alapokon került megvalósításra, ahogyan a 4.6. ábrán látható.

Az okosszerződések valósítják meg az alkalmazáshoz tartozó token kezelést, valamint a gamifikációs tranzakciók végrehajtását. Ez a megközelítés közelebb áll egy valós decentralizált rendszer működéséhez, mintha a jutalmazási logika kizárólag hagyományos szerveroldali megoldással kerülne implementálásra.

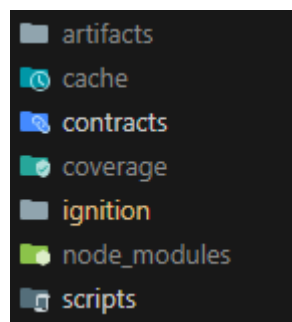
A blokklánccal való kommunikáció a frontend és a backend rétegen keresztül történik. A frontend feladata a felhasználói pénztárca csatlakoztatása, a tranzakciók kezdeményezése és a felhasználói jóváhagyások kezelése. A tranzakciók aláírása minden

esetben a felhasználó pénztárcájában történik, így az alkalmazás közvetlenül nem fér hozzá a privát kulcsokhoz.

A backend réteg közvetítő szerepet tölt be az alkalmazás és az okoszerződések között. Feladata többek között a blokklánc hívások kezelése, a tranzakciók végrehajtása, valamint bizonyos szerveroldali logikák és ellenőrzések biztosítása. A backend egy külön blokklánc pénztárcát használ az automatizált tranzakciók és a rendszerhez kapcsolódó műveletek végrehajtására.

A rendszer hibrid megközelítést alkalmaz az adattárolás során. A blokkláncon kizárólag a tokenekhez és tranzakciókhoz kapcsolódó adatok kerülnek kezelésre, míg az alkalmazás működéséhez szükséges egyéb információk, például felhasználói adatok, médiatartalmak és quest állapotok, továbbra is a relációs adatbázisban kerülnek tárolásra. Ez a megoldás lehetővé teszi a decentralizált funkcionalitás és a hatékony adatkezelés kombinálását.

A blokklánc komponensek külön projektstruktúrában kerültek elkülönítésre. A contracts könyvtár tartalmazza az okoszerződések forráskódját, míg az ignition mappa az okoszerződések blokkláncra történő telepítéséhez szükséges konfigurációkat és deployment logikát foglalja magában. A scripts könyvtár olyan segédskripteket tartalmaz, amelyek a fejlesztési műveletek támogatására szolgálnak, például backend pénztárca kezeléséhez kapcsolódóan. Az artifacts mappa a lefordított okoszerződésekhez tartozó ABI és bináris állományokat tartalmazza, amelyek szükségesek az alkalmazás és a blokklánc közötti kommunikációhoz.



4.6 ábra - Blokklánc architektúra

5. Funkcionális specifikáció és implementáció

Ebben a fejezetben a rendszer főbb funkcionalitásainak bemutatása következik. A funkcionális specifikáció célja az alkalmazás által biztosított szolgáltatások, a

felhasználói műveletek és a rendszer működésének áttekintése. A fejezet emellett az egyes funkcionalitások implementációs megvalósítását is ismerteti.

5.1 Felhasználói regisztráció / login

A rendszer lehetőséget biztosít felhasználói fiók regisztrációjára és bejelentkezésre, ahogyan az *5.1. ábrán* látható. A funkcionalitás célja az adattárolást igénylő és személyre szabott műveletek elérésének biztosítása. A fiók létrehozását követően a felhasználó jogosulttá válik médiatartalmak követésére, értékelések létrehozására, profiladatainak kezelésére, aktivitásának megtekintésére, valamint a gamifikációs rendszerhez kapcsolódó témák megszerzésére.

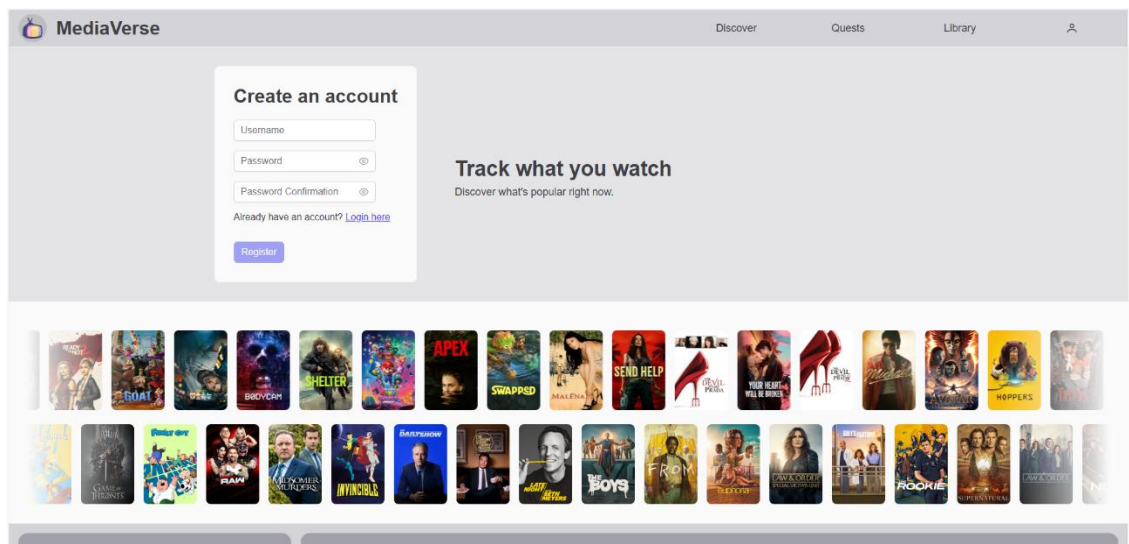
Az alkalmazás megnyitását követően a felhasználó a főoldalra kerül, ahonnan a navigációs sáv profil ikonján keresztül érhető el a bejelentkezési felület. A regisztrációs oldal a bejelentkezési felületről navigálható el.

A regisztráció során a felhasználónak meg kell adnia a felhasználónevét, jelszavát, valamint a jelszó megerősítését. A felhasználónév minimum 3 és maximum 30 karakter hosszúságú lehet. A jelszónak legalább 8 karakter hosszúnak kell lennie, továbbá tartalmaznia kell legalább egy számot, egy kisbetűt és egy nagybetűt. A jelszó megerősítésének meg kell egyeznie a megadott jelszóval.

A bejelentkezés során a felhasználónév és a jelszó megadása szükséges. A rendszer ellenőrzi a hitelesítési adatok helyességét, majd sikeres azonosítás esetén hozzáférést biztosít a védett funkciókhoz.

A felhasználói adatok biztonságos kezelését jelszó hash-elés és JWT alapú autentikáció biztosítja.

Sikeres regisztráció után a felhasználó átirányításra kerül a bejelentkezési felületre, míg sikeres bejelentkezést követően a főoldal válik elérhetővé számára.

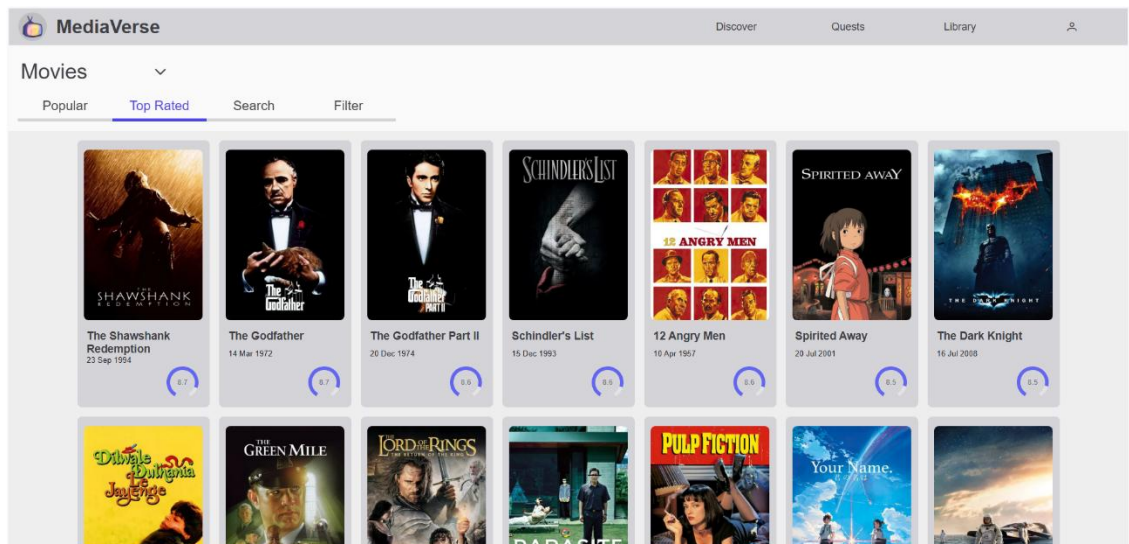


5.1 ábra - Regisztrációs felület

A regisztráció és bejelentkezés implementációja Angular oldalon service alapú HTTP kommunikációval valósult meg. Az AuthService felelős a hitelesítési végpontok meghívásáért, valamint a sikeres bejelentkezést követően a felhasználói adatok és az access token kliensoldali kezeléséért. A backend oldalon az Express routing rendszer továbbítja a kéréseket a megfelelő controller és service réteg felé, ahol megtörténik az adatok validálása, a jelszó hash-elése bcrypt használatával, valamint a JWT tokenek generálása. A refresh token HTTP-only cookie formájában kerül tárolásra a biztonságosabb autentikáció érdekében.

5.2 Média keresés és megtekintés

Az alkalmazás navigációs sávjában található Discover gombon keresztül érhető el a médiakereső felület, melyet az 5.2. ábra szemléltet. A médiakeresés lehetővé teszi a különböző médiatartalmak testreszabható böngészését és szűrését. Az alkalmazás filmek és sorozatok kezelését támogatja, így a keresési és szűrési lehetőségek is ezekhez a médiatípusokhoz igazodnak. A rendszer egyik központi funkcionálisága a médiatartalmak közötti keresés és navigáció biztosítása.



5.2 ábra - Médiakereső felület

Az oldal megnyitását követően a felhasználó kiválaszthatja a kívánt médiatípust. Ezt követően több különböző böngészési lehetőség érhető el, a legnépszerűbb vagy legjobban értékelt tartalmak listázása, kulcsszavas médiakeresés, valamint zsáner szerinti szűrés.

A legnépszerűbb és legjobban értékelt médiatartalmak megjelenítése esetén a rendszer automatikusan betölti a megfelelő adatokat. A teljesítmény optimalizálása érdekében az adatok lapozott formában (pagination) kerülnek megjelenítésre, egyszerre legfeljebb 20 elem betöltésével.

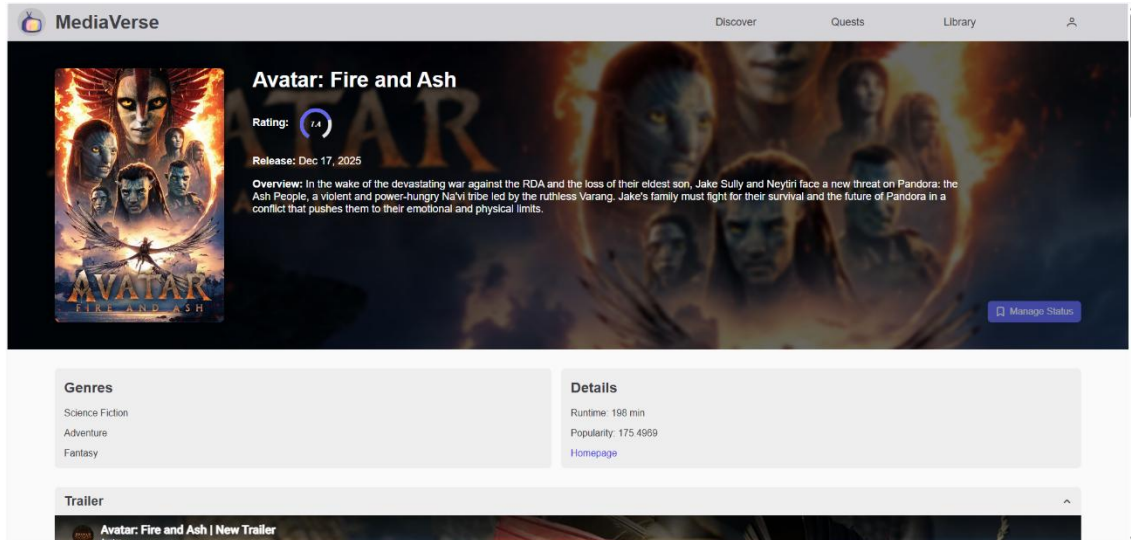
A médiakeresési funkció lehetőséget biztosít kulcsszavas keresésre. A felhasználó által megadott szöveg alapján a rendszer valós időben jeleníti meg a releváns találatokat.

A zsáner szerinti szűrés során két legördülő lista áll rendelkezésre. Az egyik a kiválasztott médiatípushoz tartozó zsánereket tartalmazza, míg a másik különböző rendezési lehetőségeket biztosít, például ábécérend szerinti növekvő vagy csökkenő rendezést.

Az adott médiatartalom kiválasztását követően megnyílik annak részletes adatlapja, ahogyan az 5.3. ábrán látható. Itt a felhasználó megtekintheti a címét, leírását, értékelését, megjelenési dátumát, zsánereit, valamint további részletes információkat, például a tartalom hosszát vagy az évadok és epizódok számát. Emellett elérhetők a felhasználói értékelések, valamint amennyiben a külső API biztosítja, előzetesek, galériaelemek és hasonló médiatartalmak is.

Bejelentkezett felhasználók számára lehetőség van a médiatartalmak követési listához adására, értékelések írására, valamint a tartalom státuszának módosítására.

A rendszer kezeli a lehetséges hibaeseteket, például a külső API hibáit, a hiányzó adatokat vagy a találat nélküli kereséseket, és minden esetben megfelelő tájékoztatást biztosít a felhasználó számára.



5.3 ábra - Média megtekintő felület

A médiakeresési funkcionalitás implementációja Angular oldalon service alapú HTTP kommunikációval történt. A különálló service-ek felelősek a médiatartalmak lekérdezéséért, kereséséért és a részletes médiaadatok betöltéséért. A kliensoldali kommunikáció Angular HttpClient segítségével valósult meg, ahol a rendszer query paraméterek formájában továbbítja a keresési és szűrési feltételeket. A kulcsszavas keresés valós idejű adatfrissítéssel működik, amely során debounce mechanizmus csökkenti a túlzott API-hívások számát. A médiaadatlapok Angular route paraméterek segítségével érhetők el.

A backend oldalon a rendszer külön service rétegen keresztül végzi a külső API-val történő kommunikációt. Az API-ból érkező válaszok feldolgozása és normalizálása biztosítja az egységes adatszerkezetet a frontend számára. A lekérdezett médiatartalmak adatbázisban kerülnek gyorsítótárazásra, így amennyiben az adatok nem elavultak, a rendszer azokat közvetlenül az adatbázisból kérdezi le. Ezáltal csökkenthető a külső API hívások száma és javítható a válaszidő. A részletes médiaadatok lekérdezése során a rendszer automatikusan feldolgozza a külső API által biztosított adatokat, például az előzeteseket, képgalériát és hasonló médiatartalmakat is.

Az értékelések kezelése külön backend végpontokon keresztül történik, ahol a rendszer biztosítja a felhasználónkénti értékelések létrehozását, módosítását és törlését.

Az értékelések mentése során a rendszer automatikus szövegszűrést alkalmaz a nem megfelelő tartalmak kiszűrésére. Az új vagy módosított értékelések dinamikusan frissülnek a média adatlapján újratöltés nélkül.

5.3 Médiakövetési lista kezelés

A médiakövetési lista az alkalmazás egyik központi funkcionálisága, amely lehetővé teszi a felhasználók számára a különböző médiatartalmak nyomon követését és rendszerezését. A bejelentkezett felhasználó a kiválasztott médiatartalmat elmentheti saját listájába, amely később a navigációs sávban található Library felületen keresztül érhető el. Az oldalon különböző szűrési és rendezési lehetőségek állnak rendelkezésre a tartalmak kezelésének megkönnyítése érdekében.

A médiatartalmak mentése a részletes médiaadatlapról történik. A hozzáadható státuszok médiatípusonként eltérnek. Filmek esetében a tervezett és befejezett állapotok érhetők el, míg sorozatoknál a tervezett, megtekintés alatt és befejezett státuszok állnak rendelkezésre. Sorozatok esetén a felhasználó megadhatja az aktuális évad- és epizódszámot is.

A sorozatok kezelésére szolgáló űrlap dinamikus működése biztosítja, hogy az évad- és epizódszám módosításával összhangban változzon a kiválasztott státusz, valamint a státusz módosításához automatikusan igazodjanak az aktuális megtekintési adatok. Ezt az 5.4. ábra szemlélteti.

5.4 ábra - Státuszkezelő űrlap

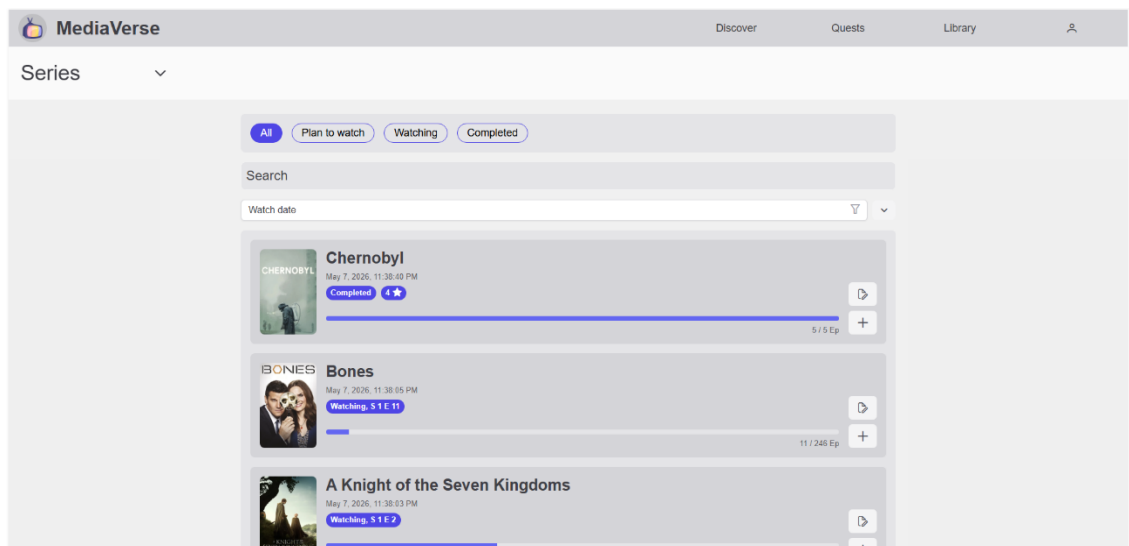
A lista felület megjelenítése során a felhasználó kiválaszthatja a kívánt médiatípust, amely alapján a rendszer lapozott formában jeleníti meg a filmeket vagy

sorozatok, ahogyan az 5.5. ábrán látható. A tartalmak státusz szerint is szűrhetők. Filmek esetében az összes, tervezett és befejezett kategóriák érhetők el, míg sorozatoknál ezek mellett a megtekintés alatt állapot is rendelkezésre áll.

A rendszer támogatja a valós idejű szöveges keresést, továbbá lehetőséget biztosít a tartalmak rendezésére megtekintési dátum, cím vagy státusz alapján növekvő és csökkenő sorrendben.

A listában megjelenített médiatartalom kiválasztásával a felhasználó elnavigálhat a részletes médiaadatlagra. Filmek esetében külön módosító felület áll rendelkezésre a státusz frissítésére, amelyen belül a mentett tartalom törlésére is lehetőség van, míg sorozatoknál ezen felül gyors epizódszám-növelési lehetőség is biztosított.

A rendszer a lehetséges hibaeseteket megfelelően kezeli, és minden esetben tájékoztatja a felhasználót az adott művelet eredményéről vagy az esetleges hibákról.



5.5 ábra - Médiakövetési felület

A médiakövetési lista kezelésének implementációja Angular oldalon külön service rétegen keresztül valósult meg. A SeriesProgressService felelős a sorozatokhoz tartozó állapotinformációk lekérdezéséért, módosításáért és törléséért HTTP alapú API-hívások segítségével. A lista lekérdezése során a kliens különböző szűrési és rendezési paramétereket továbbít a backend felé, például státusz, keresési szöveg, rendezési mező és rendezési sorrend alapján.

A backend oldalon a controller réteg kezeli a beérkező kérések validálását és a megfelelő service függvények meghívását. A service réteg végzi az üzleti logika

feldolgozását, például a státuszok ellenőrzését, az évad- és epizódszámok validálását, valamint a sorozatok megtekintési állapotának automatikus módosítását.

Az implementáció részeként megvalósításra került a gamifikációs rendszer integrációja is. A sorozatok állapotváltozásai és epizódkövetései automatikusan frissítik a felhasználóhoz tartozó küldetések állapotát, valamint a rendszer felhasználói aktivitáskövetést is végez.

A sorozat- és filmkövetési műveletek védett API végpontokon keresztül valósulnak meg. Az Express middleware réteg JWT alapú autentikációt, valamint rate limitinget alkalmaz a jogosulatlan hozzáférések és túlterhelés elleni védelem érdekében.

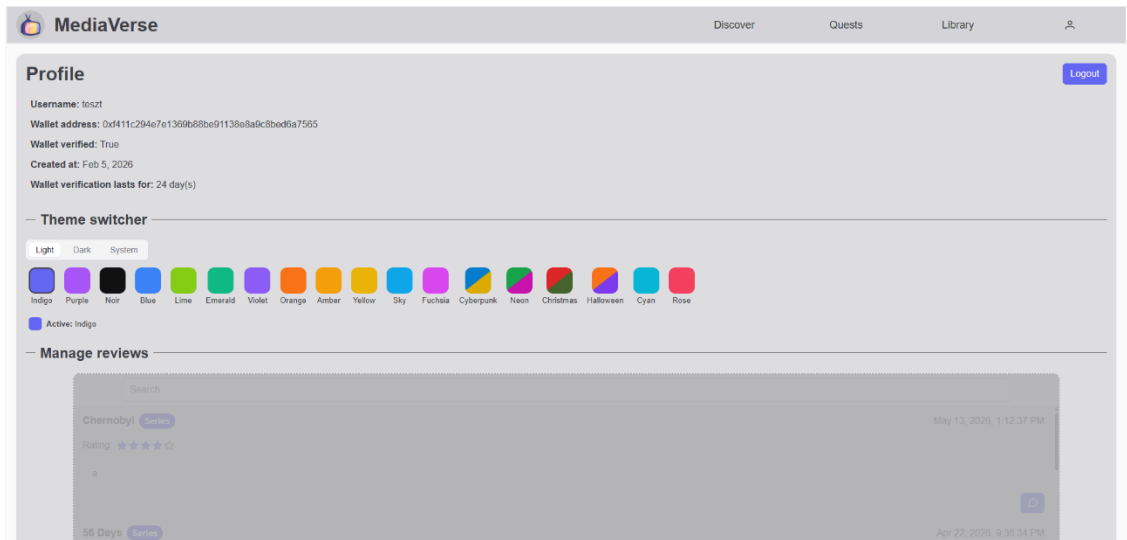
5.4 Profil adatmegjelenítés

Az 5.6. ábrán látható profilfelület célja a felhasználóhoz kapcsolódó adatok, aktivitások és statisztikák központi megjelenítése. Az oldal lehetőséget biztosít a felhasználói adatok és a médiafogyasztási adatok áttekintésére, valamint biztosítja a feladatok teljesítésén keresztül vásárolható témák váltását, és a felhasználói adatok módosítását.

A felület kizárólag bejelentkezett felhasználó számára a navigációs sávon lévő profil ikonra kattintva nyílik meg, ahol eleinte a felhasználói adatok és a kijelentkezés gomb jelennek meg. Közvetlenül alatta a téma testreszabására van lehetőség, ezen belül a világos, sötét mód és a téma váltására. Alatta a felhasználó értékeléseinek kezelése helyezkedik el, ahol lehetőség van keresni az értékelések között, módosítani, törölni az értékelést, valamint megnyitni rajta keresztül a média adatlapot. Ezt követően a felhasználói beállítások érhetők el, ezen belül név és jelszó módosítás, valamint fiók törlés lehetséges. Emellett elérhető a felhasználó elmúlt egy heti aktivitása, és médiastatisztikák.

Az alkalmazás többi részéhez hasonlóan ez a felület is dinamikus, így az adat- és értékelésmódosítás, témaváltás, felhasználói aktivitás, valamint a statisztikák mind dinamikus módon jelennek meg, így azonnali visszajelzést kap a felhasználó.

Hibakezelés felhasználói adat- és értékelésmódosítás során konzisztens, a regisztráció, bejelentkezéshez, valamint média adatlapon lévő értékeléskezeléshez hasonló a működésük.



5.6 ábra - Profil felület

A profilfelület implementációja Angular oldalon több különálló service használatával valósult meg. A UserService felelős a felhasználói adatok, aktivitások és beállítások kezeléséért, míg a UserStatisticsService biztosítja a médiastatisztikák lekérdezését. A kliensoldal HTTP alapú API-hívások segítségével kommunikál a backenddel, és a módosítások dinamikusan frissülnek az oldalon újratöltés nélkül.

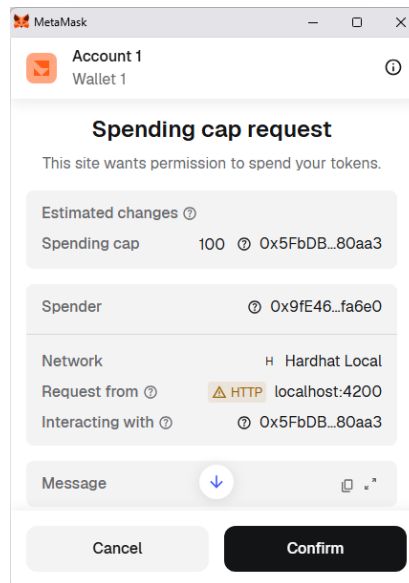
A backend oldalon a védett végpontok autentikációs middleware használatával érhetőek el, így a profilhoz kapcsolódó műveletek kizárólag bejelentkezett felhasználók számára hozzáférhetőek. A controller réteg kezeli a beérkező kérések feldolgozását és validálását, míg a service réteg végzi az üzleti logika megvalósítását, például a jelszóellenőrzést, a felhasználónév-módosítás validációját vagy az aktív téma ellenőrzését.

A felhasználói statisztikák lekérdezése külön service és DAO rétegen keresztül történik. Az adatbázis-lekérdezések aggregációs műveleteket alkalmaznak a különböző státuszok, valamint a leggyakrabban fogyasztott zsánerek meghatározására. Az implementáció során kiemelt szempont volt az érzékeny adatok védelme, ezért a rendszer a kliens felé kizárólag biztonságosan szűrt felhasználói adatokat továbbít.

5.5 Gamifikációs és jutalmazási rendszer

A feladatok felület megnyitását követően a felhasználó lehetőséget kap egy pénztárca csatlakoztatására a MetaMask szolgáltatáson keresztül. A pénztárca kapcsolódása

szükséges a gamifikációs funkciók eléréséhez, ezáltal biztosítva a jutalmazási rendszer használatát, ahogyan az 5.7. ábrán látható.



5.7 ábra - Wallet tranzakció verifikáció

Sikeres csatlakoztatást követően a korábban zárt funkciók elérhetővé válnak, emellett a pénztárca leválasztására is lehetőség van. A felhasználó ezt követően hozzáfér a feladatok kezeléséhez, azok újragenerálásához és jutalmainak begyűjtéséhez, továbbá témák vásárlásához.

A feladatok célja a felhasználói aktivitás ösztönzése különböző médiatartalmakhoz kapcsolódó kihívásokon keresztül. Egy feladat például meghatározott számú adott zsánerű médiatartalom befejezését írhatja elő. A rendszer egyszerre két aktív feladatot biztosít a felhasználó számára.

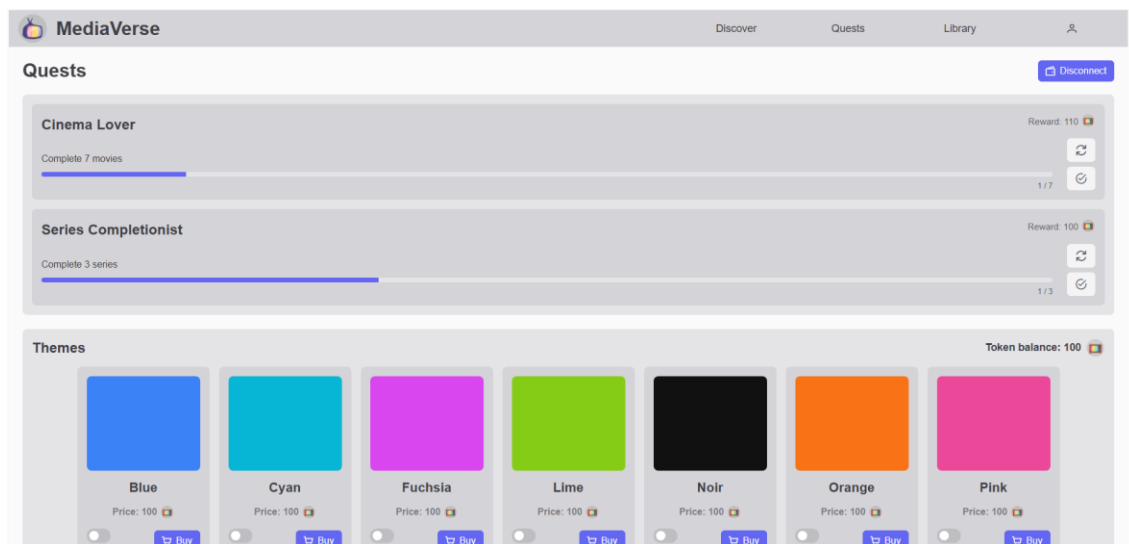
Teljesítés esetén az alkalmazás értesítést jelenít meg a felhasználó számára, valamint elérhetővé válik a jutalom begyűjtése. A művelet végrehajtását követően a rendszer a meghatározott mennyiségű token jóváírja a felhasználó számára, majd egy új feladatot generál helyette.

A felhasználó lehetőséget kap egy kiválasztott feladat újragenerálására is. Az újragenerálást követően a korábbi feladat lecserélődik, azonban a funkció ismételt használata csak huszonnégy órás visszatöltési idő letelte után válik ismét elérhetővé.

A feladatok alatt található bolt felület jeleníti meg a felhasználó aktuális tokenegyenlegét, valamint a még nem birtokolt témákat. A témák előnézetben megtekinthetők, majd megfelelő mennyiségű token birtokában megvásárolhatók.

Vásárlást követően a tokenegyenleg és az elérhető témák listája dinamikusan frissül, ahogyan az 5.8. ábrán látható.

A rendszer a feladatokhoz és vásárlásokhoz kapcsolódó műveletek során megfelelő visszajelzést biztosít a felhasználó számára, valamint kezeli a lehetséges hibaeseteket is.



5.8 ábra - Feladatok felület

A gamifikációs és jutalmazási rendszer implementációja Web3 alapú kommunikációval valósult meg. A frontend és a MetaMask pénztárca közötti kapcsolat az ethers.js könyvtár segítségével történik, amely lehetővé teszi a blokklánc alapú műveletek kezelését és az okoszerződésekkel való kommunikációt. A pénztárca csatlakoztatása során az alkalmazás digitálisan aláírandó üzenetet generál, amelynek ellenőrzésével a backend hitelesíti a felhasználó pénztárcájának tulajdonjogát.

A jutalmazási rendszer ERC-20 szabványú tokeneket kezelő okoszerződésre épül. Feladatteljesítés esetén a backend blokklánc tranzakciót kezdeményez, amely során a rendszer token jutalmat ír jóvá a felhasználó pénztárcájára. A tranzakciók végrehajtása ethers.js alapú serveroldali kommunikációval történik.

A témák vásárlása gasless tranzakciós megközelítéssel valósul meg ERC20Permit mechanizmus alkalmazásával. Ennek során a felhasználó digitális aláírással engedélyezi a tokenek felhasználását, miközben a tranzakció hálózati költségét a backend kezeli. A ThemeMarketplace okoszerződés ellenőrzi a tokenegyenleget, végrehajtja a tokenek elégetését, majd rögzíti a megvásárolt témát. Az okoszerződés vásárlási logikájának egy részlete az alábbi 5.9. ábrán látható.

```
ThemeMarketplace.sol

function purchaseThemeForWithPermit(
    address user,
    string memory themeId,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external onlyOwner nonReentrant {
    uint256 price = themePrices[themeId];
    require(price > 0, "Theme does not exist");
    require(!userThemes[user][themeId], "Theme already owned");
    require(token.balanceOf(user) ≥ price, "Insufficient token balance");

    IERC20Permit(address(token)).permit(
        user,
        address(this),
        price,
        deadline,
        v, r, s
    );

    ERC20Burnable(address(token)).burnFrom(user, price);

    userThemes[user][themeId] = true;
    emit ThemePurchased(user, themeId, price);
}
```

5.9 ábra - ERC20Permit alapú témavásárlási logika

A rendszer több biztonsági ellenőrzést is alkalmaz a gamifikációs funkciók során. A backend ellenőrzi a pénztárca hitelességét, a tokenegyenleget, a felhasználó tulajdonában lévő témákat, valamint a feladatok újragenerálására vonatkozó visszatöltési időt is.

6. Tesztelés

A rendszer megbízható működésének ellenőrzése érdekében több különböző tesztelési módszer került alkalmazásra. A frontend és backend egyes komponenseinek működését unit tesztek vizsgálták, míg a backend REST API végpontjai külön API tesztelésen estek át. A szerveroldali teljesítmény vizsgálata k6 alapú terheléses tesztekkel történt. Emellett a blokklánc alapú funkcionalitások esetében okoszerződés unit tesztek, valamint különböző blokklánc hálózatok összehasonlító vizsgálata is megvalósult.

6.1 Unit tesztek

Az alkalmazás kliensoldali, szerveroldali és blokklánc alapú részeihez unit tesztek készültek. Kliensoldalon Angular környezetben Jasmine és Karma használatával

készültek tesztek az autentikációs service-re és a login komponensre. Szerveroldalon Jest alapú tesztek íródtak a quests service és a users service működésének ellenőrzésére. A blokklánc rétegben az okosszerződések működésének biztosítása érdekében szintén unit tesztek készültek.

A kliensoldali login komponens tesztelése során ellenőrzésre került az űrlap inicializálása, a validációs szabályok működése, az üres mezők kezelése, valamint a sikeres és sikertelen bejelentkezési folyamatok. A tesztek vizsgálták továbbá a töltési állapot működését, a megfelelő navigációt és az értesítések megjelenítését is. Az autentikációs service tesztjei során a bejelentkezési, regisztrációs, kijelentkezési és tokenfrissítési folyamatok működése került ellenőrzésre. Emellett vizsgálatra került a HTTP-kérések helyessége, a tokenek tárolása és törlése, a felhasználói adatok változása, valamint a témabeállítások alkalmazása is. A frontend tesztek során a HTTP kommunikáció mockolt HttpClient használatával történt, így nem volt szükség valós backend kapcsolatra.

Szerveroldalon Jest alapú unit tesztek készültek, ahol a külső függőségek mockolásra kerültek, így a tesztek kizárólag az üzleti logika működésére fókuszáltak. A users service tesztjei során ellenőrzésre került a regisztráció működése, a jelszóvalidáció, a login folyamat, a token generálás, a kijelentkezés, a felhasználónév- és jelszómódosítás, valamint a hibakezelési esetek. A quests service tesztjei lefedték a küldetések lekérését, az újragenerálási folyamatot, a visszatöltési rendszer működését, a jutalmak igénylését, a feladatokban való haladás növelését, valamint a csalás elleni ellenőrzéseket, ahogyan a 6.1. ábrán látható.

```
Test Suites: 1 passed, 1 total
Tests:      28 passed, 28 total
Snapshots:  0 total
Time:       9.081 s
Ran all test suites matching quests.service.test.js.
```

6.1 ábra - Quests service unit teszteredmény

A blokklánc alapú unit tesztek Hardhat környezetben készültek. A MediaVerseToken szerződés tesztjei ellenőrizték a tokenkészlet helyességét, a jutalmazási mechanizmust, az egyenlegek változását, a jogosultságkezelést, valamint a hibás tranzakciók kezelését. A ThemeMarketplace szerződés tesztjei vizsgálták a témavásárlási folyamatot, az ERC-20 permit alapú tranzakciókat, a témák

tulajdonjogának ellenőrzését, az adminisztrátori funkciókat, valamint a különböző hibakezelési eseteket.

6.2 API tesztelés

Az API végpontok tesztelésére Postman Collection készült. A collection struktúrája a backend modulfelépítését követi. Az endpointok külön mappákba vannak szervezve, például users, movies, movie-progress, user-reviews, és egyéb modulok szerint.

Minden végponthoz külön leírás tartozik, ami tartalmazza az endpoint célját, hogy szükséges-e autentikáció, valamint a végpont működésének rövid összefoglalását.

A bejelentkezési végpont automatikusan eltárolja a visszakapott JWT access token. Ha az access token érvényessége lejár, a frissítési útvonal által adott token szintén automatikusan eltárolódik. A token Postman collection változóként kerül mentésre. Ennek köszönhetően a védett végpontok manuális tokenmásolás nélkül tesztelhetők. A token kezelés automatizálásával a tesztelési folyamat gyorsabbá és kényelmesebbé vált.

A végpontok előre definiált tesztadatokat is tartalmaznak, amelyek szabadon módosíthatók különböző bemeneti esetek vizsgálatához. A collection támogatja query paraméterek, request body-k, pagination, filtering, sorting, search funkciók tesztelését.

A blokklánc alapú végpontok EIP-712 aláírást igényelnek, ezek közvetlen Postman tesztelése korlátozott, ezért ezen végpontok dokumentálva lettek, de nem futtathatók.

A collection működésének bemutatására az alábbiakban a /users/login végpont tesztelési folyamata kerül ismertetésre. A /users/login végpont tesztelése során JSON formátumban felhasználónév és jelszó kerül elküldésre POST kérésként. A backend JWT access tokent küld vissza, amelyet a Postman automatikusan eltárol collection változóban. Hiba vagy nem létező felhasználónév és jelszó esetén a megfelelő hibaüzenettel tér vissza a végpont.

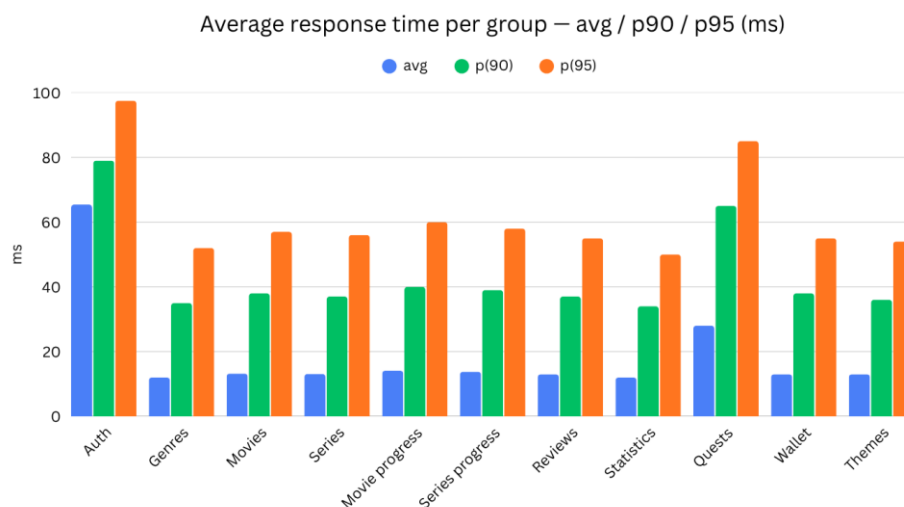
6.3 Teljesítményteszt

A szerveroldali teljesítménytesztelés k6 terhelésvizsgáló eszközzel, Node.js környezetben történt. A blokklánc végpontok tesztelése bypass módban valósult meg, így a backend végpontok valódi működése futott le, azonban a permit alapú tranzakciók teljes végrehajtása nem történt meg a frontend wallet interakció szükségessége miatt.

A teszt célja annak vizsgálata volt, hogy az alkalmazás hogyan viselkedik növekvő terhelés mellett, különös tekintettel a válaszidőkre, hibaarányokra, hitelesítési folyamatokra és blokklánc integrációkra. A terheléses vizsgálat során a virtuális felhasználók száma fokozatosan 10-ről 100-ra emelkedett.

A teszt során meghatározott küszöbértékek alapján az összes HTTP kérés 95%-ának válaszideje 2000 ms alatt kellett maradjon, míg a hibaarány nem haladhatta meg a 10%-ot.

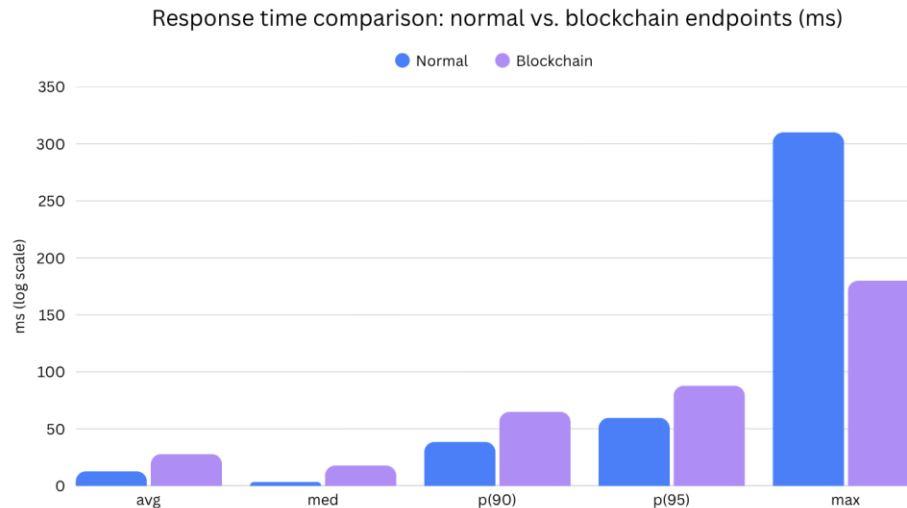
A 6.2. ábra az egyes végpontcsoportok átlagos, 90. percentilis és 95. percentilis válaszidejét mutatja. Megfigyelhető, hogy a hitelesítési folyamatok rendelkeznek a legmagasabb válaszidővel, ami elsősorban a jelszó hash-elési és tokenkezelési műveletek következménye. A blokklánc alapú küldetések szintén magasabb válaszidőt mutattak, azonban még ezek esetében is 100 ms alatt maradt a 95. percentilis érték. A legtöbb hagyományos API végpont válaszideje stabilan 50-60 ms közötti p95 értéken maradt, ami megfelelő teljesítményt jelez még magasabb terhelés mellett is.



6.2 ábra - Átlagos válaszidő diagram

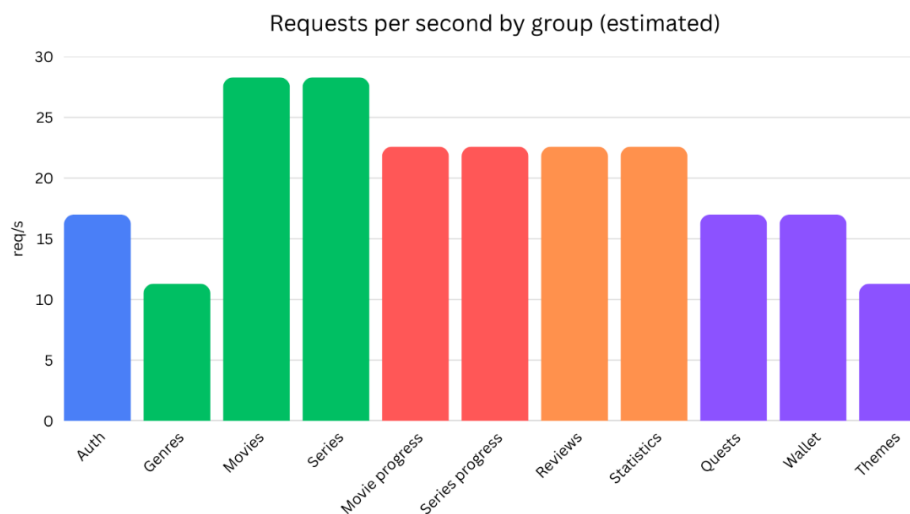
A 6.3. ábra a hagyományos és blokklánc integrációt használó végpontok válaszidejének összehasonlítását mutatja. A blokklánc funkciókhoz kapcsolódó végpontok átlagosan magasabb válaszidővel rendelkeztek, ami a további validációs és aláírás-ellenőrzési folyamatokból adódik. Ennek ellenére a válaszidők továbbra is stabil tartományban maradtak, és nem jelentkezett kritikus teljesítményromlás. Megfigyelhető ugyanakkor, hogy a hagyományos végpontok maximális válaszideje magasabb volt, mint a blokklánc alapú végpontoké. Ez arra utal, hogy bár a blokklánc műveletek átlagosan

lassabbak, válaszidejük egyenletesebb és kiszámíthatóbb maradt a terhelés során. A hagyományos végpontok esetében egyes kiugró terhelési helyzetek nagyobb válaszido-ingadozást eredményeztek, ami magasabb maximális értékekhez vezetett.



6.3 ábra - Hagyományos és blokklánc végpont válaszido diagram

A 6.4. ábra az egyes végpontcsoportok becsült másodpercenkénti kéréskezelési képességét szemlélteti. A hagyományos tartalomlekérő végpontok, például a filmekkel és sorozatokkal kapcsolatos API-k, közel 28 kérés/másodperc teljesítményt értek el. A blokklánc műveletekhez kapcsolódó végpontok alacsonyabb értékeket mutattak, azonban ezek összetettebb működése miatt ez várható eredménynek tekinthető. A mért adatok alapján a rendszer megfelelő stabilitással és válaszkészséggel működött a vizsgált terhelési tartományban.



6.4 ábra - Másodpercenkénti kéréskezelési diagram

6.4 Blokklánc hálózatok összehasonlítása

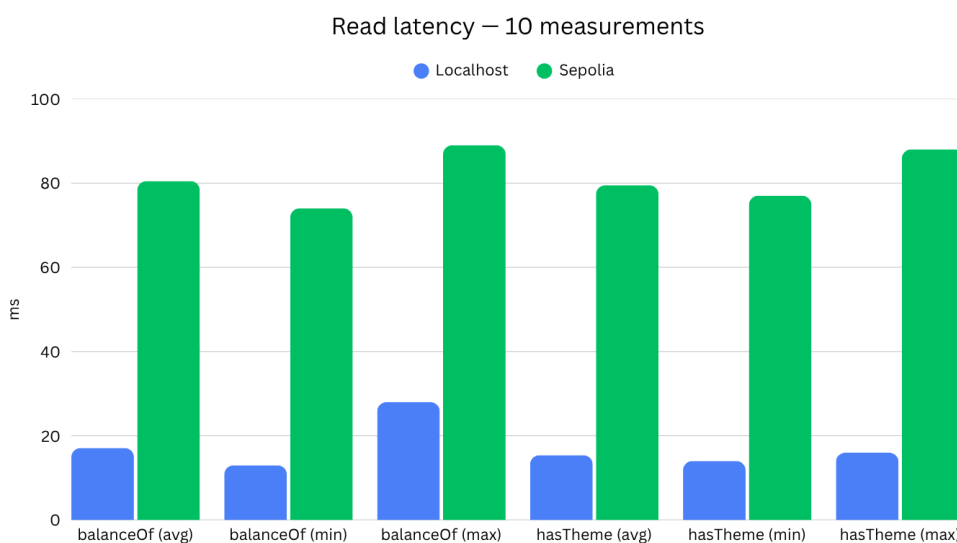
A blokklánc integráció működésének vizsgálatához összehasonlító tesztek készültek lokális Hardhat hálózaton és Ethereum Sepolia teszhálózaton. A mérések célja a válaszidők, blokkgenerálási sajátosságok és gas becslések összevetése volt.

A tesztek ethers.js használatával történtek, közvetlenül az okosszerződések hívásán keresztül. A vizsgálat kiterjedt az olvasási műveletek válaszidejére (balanceOf, hasTheme), a blokkgenerálási időkre, valamint az írási tranzakciók várható gas fogyasztására.

A mérések alapján a localhost hálózat átlagosan 17 ms válaszidőt biztosított, a Sepolia hálózat átlagosan 78 ms válaszidőt produkált, a Sepolia átlagos blokkideje körülbelül 12 másodperc volt, a gas becslések közötti eltérés 1% alatti maradt.

A 6.5. ábra a balanceOf és hasTheme lekérdezések minimum, átlagos és maximális válaszidejét mutatja lokális és Sepolia hálózaton.

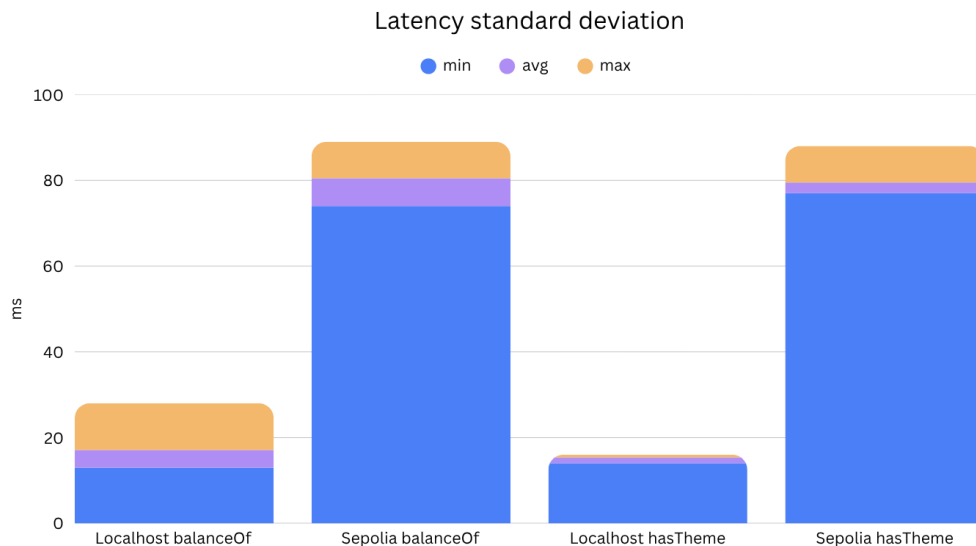
A lokális környezet minden mérésben jelentősen alacsonyabb válaszidőt biztosított, mivel a blokklánc lokálisan futott, hálózati kommunikáció nélkül. A Sepolia hálózat esetén a válaszidőket a külső RPC kommunikáció és a publikus Ethereum infrastruktúra késleltetése növelte. Ennek ellenére a válaszidők stabilnak tekinthetők, mivel az egyes mérések közötti eltérés alacsony maradt.



6.5 ábra - Olvasási művelet válaszidő diagram

A 6.6. ábra a localhost és a Sepolia hálózaton mért válaszidők minimum, átlagos és maximális értékeinek egymáshoz viszonyított eltérését szemlélteti. A diagram egymásra épülő oszlopokkal jeleníti meg a minimum, átlagos és maximális válaszidőket, ezáltal jól láthatóvá válik az egyes mérések közötti ingadozás mértéke.

A localhost környezetben mind a balanceOf, mind a hasTheme lekérdezések alacsonyabb válaszidőkkel működtek, és a minimum, valamint maximális értékek közötti eltérés is kisebb maradt. A Sepolia hálózaton magasabb válaszidők voltak megfigyelhetők, amely elsősorban a külső RPC kommunikációból és a publikus Ethereum infrastruktúrából eredő késleltetés következménye. Ennek ellenére a mérések közötti eltérés nem volt jelentős, így a válaszidők stabilnak és kiszámíthatónak tekinthetők.

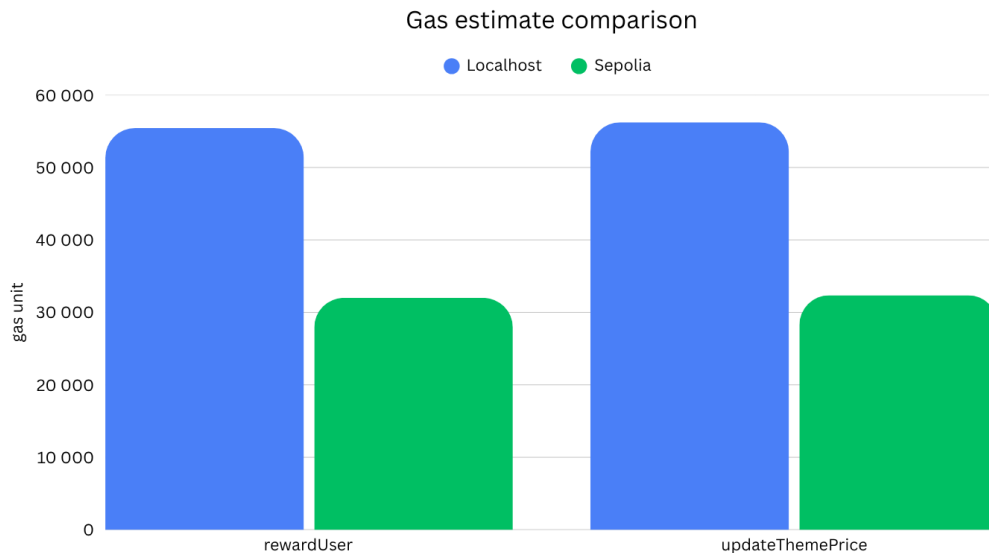


6.6 ábra - Válaszidő szórás diagram

A 6.7. ábra a rewardUser és updateThemePrice tranzakciók becsült gas költségeit hasonlítja össze localhost és Sepolia hálózaton.

A két hálózat között minimális eltérés volt megfigyelhető a gas becslésekben. Ez várható eredmény, mivel azonos okoszerződés bytecode futott mindkét környezetben.

A mérés igazolja, hogy a lokális fejlesztői környezet megfelelően reprezentálja a publikus Ethereum teszhálózat működését a tranzakciós költségek szempontjából.



6.7 ábra - Gas becslés összehasonlító diagram

7. Eredmények és értékelések

7.1 Rendszer működése

A rendszer által biztosított fő funkciók közé tartozik a médiatartalmak böngészése, a tartalmak felhasználói fiókhoz rendelése különböző státuszokkal, az értékelések létrehozása, a felhasználói aktivitás követése, valamint a gamifikációs elemekre épülő tématestreszabási lehetőségek biztosítása.

A tervezett funkcionalitások sikeresen megvalósításra kerültek, így az alkalmazás egy stabilan működő, kényelmesen használható rendszert biztosít. A kliensoldal, a szerveroldali API, az adatbázis és a blokklánc komponensek közötti kommunikáció megfelelően működik, ezáltal a rendszer képes a felhasználói műveletek valós idejű kezelésére és a médiaadatok hatékony feldolgozására.

A fejlesztés során kiemelt szempont volt a modularitás és a bővíthetőség biztosítása. Az alkalmazás architektúrája lehetővé teszi további médiatípusok, új gamifikációs elemek vagy további blokklánc funkciók későbbi integrálását is.

7.2 Teljesítmény

Az alkalmazás teljesítménye a tesztelések során stabilnak bizonyult. A válaszidők kis és közepes terhelés mellett is megfelelő tartományban maradtak, miközben a hibaarány nem

növekedett kritikus mértékben. A terheléses vizsgálatok alapján a rendszer képes volt több egyidejű felhasználói kérés kezelésére is nagyobb teljesítményromlás nélkül.

A gyorsítótárazási mechanizmusok alkalmazása javította a médiatartalmak lekérdezési sebességét, valamint csökkentette a külső API-hívások számát. Ennek köszönhetően a gyakran lekérdezett adatok gyorsabban elérhetővé váltak, miközben a rendszer terhelése is mérséklődött.

A blokklánc alapú funkciók válaszideje a hagyományos végpontokhoz képest várhatóan magasabb volt a decentralizált működésből adódó többlet kommunikáció és validáció miatt. Ennek ellenére a blokklánc integráció stabilan működött, és a mért válaszidők a tesztelési környezetben elfogadható tartományban maradtak.

7.3 Korlátok

Az alkalmazás fejlesztése során több technikai és működésbeli korlát is megjelent, amelyek befolyásolták a rendszer kialakítását és működését.

Az egyik legjelentősebb korlátot a külső API-függőség jelenti. A külső médiaadatbázis-szolgáltatás elérhetősége, válaszideje, rate limit korlátozása, valamint az adatok minősége közvetlen hatással lehet az alkalmazás működésére és a médiaadatok lekérdezésére.

A blokklánc alapú funkciók használata magasabb válaszidővel járhat, különösen publikus Ethereum hálózat használata esetén. A tranzakciók véglegesítése függ a hálózat aktuális terhelésétől és a blokkgenerálási időtől, ezért a blokklánc műveletek sebessége kevésbé kiszámítható, mint a hagyományos backend műveleteké.

További korlátot jelent a MetaMask pénztárca használatának szükségessége. A wallet integráció külső szolgáltatásra épül, ezért annak működése és felhasználói felülete nem szabható teljes mértékben tesztre. Emellett a blokklánc alapú rendszerek használata bizonyos felhasználók számára bonyolultabb lehet, különösen azok esetében, akik nem rendelkeznek korábbi tapasztalattal kriptovaluta pénztárcák használatában.

7.4 Továbbfejlesztési lehetőségek

A rendszer továbbfejlesztési lehetőségei között szerepel a támogatott médiatípusok bővítése. A jelenlegi film- és sorozatkezelés mellett integrálhatóvá válhatnak további

külső API-k, amelyek például könyvek, képregények, videojátékok vagy egyéb médiatartalmak kezelését tennék lehetővé.

További irányt jelenthet a felhasználói élmény bővítése progresszív webalkalmazás (PWA) támogatással, amely lehetővé tenné az alkalmazás telepíthetőségét mobil eszközökön.

Emellett a rendszer kiegészíthető további közösségi funkciókkal is, például felhasználók közötti kapcsolatfelvétellel, illetve profilok megtekintésével.

8. Összefoglalás

A szakdolgozat célja egy olyan fullstack webalkalmazás megtervezése és megvalósítása volt, amely lehetőséget biztosít különböző médiatartalmak egységes kezelésére, nyomon követésére és értékelésére. A fejlesztés során sikerült egy olyan rendszer létrehozása, amely ötvözi a modern webes technológiákat a blokklánc alapú gamifikációs megoldásokkal.

A projekt fejlesztése során tapasztalatot szereztem frontend és backend technológiák alkalmazásában, rendszerarchitektúra tervezésében, adatbázis-kezelésben, valamint külső API-k integrációjában. Emellett lehetőségem nyílt a Web3 technológiák és az okoszerződések működésének megismerésére is, különös tekintettel az Ethereum alapú tranzakciókezelésre és az ERC-20 szabvány alkalmazására.

A megvalósított rendszer stabil alapot biztosít további funkciók és médiatípusok integrálásához, így a projekt a későbbiekben tovább bővíthető és fejleszthető.

Irodalomjegyzék

- [1] „Recent updates | Goodreads”. Elérés: 2026. május 18. [Online]. Elérhető: <https://www.goodreads.com/>
- [2] „The Biggest Video Game Database on RAWG - Video Game Discovery Service”. Elérés: 2026. május 18. [Online]. Elérhető: <https://rawg.io/>
- [3] „Letterboxd • Your life in film”. Elérés: 2026. május 18. [Online]. Elérhető: <https://letterboxd.com/>
- [4] „MyAnimeList.net - Anime and Manga Database and Community”, MyAnimeList.net. Elérés: 2026. május 18. [Online]. Elérhető: <https://myanimelist.net/>
- [5] R. Lobato és A. Lotz, „Beyond Streaming Wars: Rethinking Competition in Video Services”, *Media Industries Journal*, köt. 8, sz. 1, okt. 2021, doi: [10.3998/mij.1338](https://doi.org/10.3998/mij.1338).
- [6] „Frequent questions”. Elérés: 2026. május 18. [Online]. Elérhető: <https://letterboxd.com/about/faq/>
- [7] „Film data”. Elérés: 2026. május 18. [Online]. Elérhető: <https://letterboxd.com/about/film-data/>
- [8] „Trakt Web: About”, Trakt Web. Elérés: 2026. május 18. [Online]. Elérhető: <https://app.trakt.tv/about>
- [9] „About TMDb — The Movie Database (TMDb)”. Elérés: 2026. május 19. [Online]. Elérhető: <https://www.themoviedb.org/about>
- [10] „Getting Started”, The Movie Database (TMDb). Elérés: 2026. május 19. [Online]. Elérhető: <https://developer.themoviedb.org/docs/getting-started>
- [11] S. Deterding, D. Dixon, R. Khaled, és L. Nacke, „From Game Design Elements to Gamefulness: Defining Gamification”, előadás Proceedings of the 15th International Academic MindTrek Conference: Envisioning Future Media Environments, MindTrek 2011, szept. 2011, o. 9–15. doi: [10.1145/2181037.2181040](https://doi.org/10.1145/2181037.2181040).
- [12] Z. Zainuddin, S. K. W. Chu, M. Shujahat, és C. J. Perera, „The impact of gamification on learning and instruction: A systematic review of empirical evidence”, *Educational Research Review*, köt. 30, o. 100326, jún. 2020, doi: [10.1016/j.edurev.2020.100326](https://doi.org/10.1016/j.edurev.2020.100326).
- [13] J. H. Witte, „The Blockchain: A Gentle Four Page Introduction”, 2016. december 6., *arXiv:arXiv:1612.06244*. doi: [10.48550/arXiv.1612.06244](https://doi.org/10.48550/arXiv.1612.06244).
- [14] D. B. Rawat, V. Chaudhary, és R. Doku, „Blockchain: Emerging Applications and Use Cases”, 2019. április 28., *arXiv:arXiv:1904.12247*. doi: [10.48550/arXiv.1904.12247](https://doi.org/10.48550/arXiv.1904.12247).
- [15] A. Team, „What is Angular? • Angular”. Elérés: 2026. május 19. [Online]. Elérhető: <https://angular.dev/overview>
- [16] „The starting point for learning TypeScript”. Elérés: 2026. május 19. [Online]. Elérhető: <https://www.typescriptlang.org/docs/>
- [17] „SPA (Single-page application) - Glossary | MDN”, MDN Web Docs. Elérés: 2026. május 19. [Online]. Elérhető: <https://developer.mozilla.org/en-US/docs/Glossary/SPA>
- [18] „Introduction to Node.js | Node.js Learn”. Elérés: 2026. május 19. [Online]. Elérhető: <https://nodejs.org/learn/getting-started/introduction-to-nodejs>
- [19] „Express.js · Node.js web application framework”, Express.js. Elérés: 2026. május 19. [Online]. Elérhető: <https://expressjs.com/en/>
- [20] „REST - Glossary | MDN”, MDN Web Docs. Elérés: 2026. május 19. [Online]. Elérhető: <https://developer.mozilla.org/en-US/docs/Glossary/REST>
- [21] „PostgreSQL: About”. Elérés: 2026. május 2. [Online]. Elérhető: <https://www.postgresql.org/about/>
- [22] „Relational Database: Structure, Benefits, and Key Use Cases”. Elérés: 2026. május 2. [Online]. Elérhető: <https://www.accedata.io/blog/what-is-a-relational-database-architecture-features-and-real-world-applications>
- [23] D. Quilty, „Choosing the Right Database: MariaDB vs. MySQL, PostgreSQL, and MongoDB”, Percona. Elérés: 2026. május 2. [Online]. Elérhető: <https://www.percona.com/blog/choosing-the-right-database-comparing-mariadb-vs-mysql-postgresql-and-mongodb/>
- [24] M. Biehl, *API Architecture*. API-University Press, 2015.
- [25] „REST API basics and implementation”, Google Cloud. Elérés: 2026. május 18. [Online]. Elérhető: <https://cloud.google.com/discover/what-is-rest-api>
- [26] „FAQ”, The Movie Database (TMDb). Elérés: 2026. május 18. [Online]. Elérhető: <https://developer.themoviedb.org/docs/faq>
- [27] S. C. Team, „Blockchain Hashing: Ensuring Security and Immutability”, Shardeum | India’s EVM-Compatible Layer 1 Blockchain. Elérés: 2026. május 3. [Online]. Elérhető: <https://shardeum.org/blog/blockchain-hashing/>
- [28] „What is Ethereum? (A Complete Guide)”, ethereum.org. Elérés: 2026. május 3. [Online]. Elérhető: <https://ethereum.org/what-is-ethereum/>
- [29] „Ethereum applications | Decentralized Applications on Ethereum”, ethereum.org. Elérés: 2026. május 3. [Online]. Elérhető: <https://ethereum.org/what-are-apps/>

- [30] „Getting started with Hardhat 3”, Hardhat 3. Elérés: 2026. május 3. [Online]. Elérhető: <https://hardhat.org/docs/getting-started>
- [31] „Databases in the Blockchain Era Will blockchain technology replace traditional databases, or is there a more complex relationship? Discover how blockchain and databases might coexist, compete, or merge in the evolving data landscape.” Elérés: 2026. május 3. [Online]. Elérhető: <https://www.rapydo.io/blog/databases-in-the-blockchain-era>
- [32] „Tokenize.it | What is a wallet and why do I need a private key?” Elérés: 2026. május 4. [Online]. Elérhető: <https://tokenize.it/en/resources/blog/wallet>
- [33] „FAQs | MetaMask - What is MetaMask, Supported Coins, Wallet Address”. Elérés: 2026. május 4. [Online]. Elérhető: <https://metamask.io/faqs>
- [34] „ERC-20 Token Standard”, ethereum.org. Elérés: 2026. május 19. [Online]. Elérhető: <https://ethereum.org/developers/docs/standards/tokens/erc-20/>
- [35] „ERC-721 Non-Fungible Token Standard”, ethereum.org. Elérés: 2026. május 19. [Online]. Elérhető: <https://ethereum.org/developers/docs/standards/tokens/erc-721/>
- [36] „ERC-1155 Multi-Token Standard”, ethereum.org. Elérés: 2026. május 19. [Online]. Elérhető: <https://ethereum.org/developers/docs/standards/tokens/erc-1155/>
- [37] „How to copy your address and view account details | MetaMask Help Center”. Elérés: 2026. május 4. [Online]. Elérhető: [https://support.metamask.io/configure/wallet/how-to-copy-your-metamask-account-public-address-/](https://support.metamask.io/configure/wallet/how-to-copy-your-metamask-account-public-address/)
- [38] „What is a token approval? | MetaMask Help Center”. Elérés: 2026. május 4. [Online]. Elérhető: <https://support.metamask.io/stay-safe/safety-in-web3/what-is-a-token-approval/>

Nyilatkozat

Alulírott Zoltai Zétény Csongor, programtervező-informatikus BSc szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztési Tanszékén készítettem, programtervező-informatikus diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy szakdolgozatomat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026.05.21. Zoltai Zétény Csongor

Köszönetnyilvánítás

Szeretnék köszönetet mondani Dr. Pflanzner Tamás témavezetőmnek a szakmai segítségéért, támogatásáért és hasznos tanácsaiért.